



UNIVERZITET U TUZLI
FAKULTET ELEKTROTEHNIKE
AUTOMATIKA I ROBOTIKA

ZAVRŠNI RAD:

Realizacija ball beam sistema i
upravljanje koristeći linearne i nelinearne
metode upravljanja

Student:
NURA EFENDIĆ

Mentor: dr. sci.
Jakub OSMIĆ, red. prof.

Tuzla, Septembar 2024

Sadržaj

1	Sažetak	3
2	Abstract	4
3	Uvod	5
4	Matematički model	6
4.1	Lagrangeova metoda	6
4.2	Linearizacija sistema u prostoru stanja	8
5	Kontroleri	10
5.1	Kompenzator sa faznim prednjačenjem	11
5.1.1	Matlab kod	14
5.1.2	Simulink	15
5.2	Kontroler sa povratnom spregom po cijelom vektoru stanja	17
5.2.1	Matlab kod i analiza rezultata	18
5.2.2	Dodavanje integratora u sistem sa povratnom spregom stanja	20
5.2.3	Izračunavanje povratne sprege stanja K	20
5.2.4	Simulink	23
5.2.5	Implementacija i analiza kontrolera sa povratnom spregom po cijelom vektoru stanja na realnom sistemu	24
5.2.6	Unaprijedna kontrola: kod za ESP32	26
5.3	PID kontroler	30
5.3.1	Matlab kod i analiza rezultata	31
5.3.2	Simulink	34
5.3.3	Implementacija i analiza PID kontrolera na realnom sistemu	36
5.4	Fuzzy kontroler	42
5.4.1	Kod za ESP32	46
5.4.2	Analiza rezultata fuzzy kontrolera na realnom modelu	50
6	Tehnička implementacija modela	51
6.1	Korištene komponente	51
6.2	Očitavanje pozicije pomoću SoftPot linearnog membranskog potencijometra	52
6.3	Električna šema	53
6.4	Realizacija sistema	54
7	Prikupljanje podataka	55
7.1	Kod na ESP32	55
7.2	Server kod	55
8	Zaključak	58

Popis slika

1	Šema dijagrama sistema lopte i grede	7
2	GMK sistema i prostor koji zadovoljava zahtjeve preskoka i vremena smirenja .	11
3	GMK sistema sa dodatim kompenzatorom, $z_0 = 0.01$ $p_0 = 7$	12
4	Križićima označeni polovi na GMK	13
5	Odziv na step sistema sa kompenzatorom $C(s)$	13
6	Model sistema u simulink-u	15
7	Model sistema sa dodanim kompenzatorom	15
8	Odziv sistema sa kompenzatorom sa faznim prednjačenjem	16
9	Šema sistema sa povrtanom spregom po vektoru stanja	17
10	Odziv sistema sa kontrolerom sa povratnom spregom po vektoru stanja	19
11	Odziv sistema sa kontrolerom sa povratnom spregom po vektoru stanja sa uklonjenom greškom u stacionarnom stanju	19
12	Odziv sistema sa kontrolerom sa povratnom spregom po vektoru stanja uz dodani integrator u matlabu za linearizirani model	22
13	Nelinearni model sistema u simulinku sa povratnom vezom po vektoru stanja . .	23
14	Odziv nelinearnog modela sistema u Simulink-u sa povratnom vezom po vektoru stanja i dodanim integratorom greške pozicije	23
15	Odviz sistema sa povratnom spregom po vektoru stanja	25
16	Odziv sistema sa kontrolerom sa povratnom spregom vektora stanja uz dodani integrator $r_{ref} = 20$	29
17	Blok dijagram sistema zadatog prenosnom funkcijom $G(s)$ sa kontrolerom $C(s)$.	30
18	Odziv sistema sa PID kontrolerom sa samo sliciproporcionalnom komponentom .	32
19	Odziv sistema kontrolisan PID kontrolerom sa proporcionalnom i derivacionom komponentom	32
20	Odziv sistema kontrolisam PID kontrolerom za $K_p = 15$ i $K_d = 15$	33
21	Odziv sistema sa PID kontrolerom $K_p = 15$ $K_d = 40$ $K_i = 0.2$	33
22	PID kontroler primjenjen na nelinearni model	34
23	Odziv sistema sa PID kontrolerom $T_s = 15.96$ $MPOS = 7\%$ $K_p = 0.4$ $K_i = 0.003$ $K_d = 1.2$	35
24	Odziv realnog sistema sa pojačanjima $K_p = 15$ $K_d = 40$ $K_i = 0.2$	38
25	Odziv realnog sistema sa PID kontrolerom uz pocetne uslove $r(0) = 33$	39
26	$K_p = 0.4$ $K_i = 0.003$ $K_d = 1.2$	41
27	Funkcije prirodnosti ulaza za $r_{ref} = 20$	43
28	Funkcije pripadnosti izlaza	43
29	Vizuelni prikaz veze ulaza i izlaza - Fuzzy upravljanje	45
30	Odziv sistema upravljani fuzzy kontrolerom	50
31	Korištene komponente	51
32	Linearni potencijometar	52
33	Električna šema	53
34	Realizovani model	54

1 Sažetak

Ovaj rad se bavi realizacijom i analizom upravljanja ball and beam sistemom koristeći različite linearne i nelinearne metode upravljanja, uključujući PID kontroler, fuzzy kontroler, te kontroler sa povratnom spregom po cijelom vektoru stanja. MATLAB je korišten za matematičko modeliranje sistema, simulacije kontrolera, te analizu performansi prije njihove implementacije na stvarnom sistemu.

Nakon modeliranja sistema koristeći Lagrangeovu metodu, izvršena je linearizacija i dizajn kontrolera. Različiti kontroleri su simulirani u MATLAB-u kako bi se optimizovali parametri i predvidjele performanse sistema. Kontroleri su zatim implementirani na fizički model uz pomoć ESP32 mikrokontrolera.

Eksperimentalni rezultati pokazuju da svaki kontroler ima specifične prednosti zavisno od kriterija kao što su vrijeme smirenja, maksimalni preskok i greška u stacionarnom stanju.

2 Abstract

This thesis focuses on the implementation and analysis of control strategies for a ball and beam system using various linear and nonlinear control methods, including PID control, fuzzy controller, and full-state feedback control. MATLAB was used for mathematical modeling, controller simulations, and performance analysis before applying these controllers to the actual system.

The system was first modeled using the Lagrange method, followed by linearization and controller design. Various controllers were simulated in MATLAB to optimize parameters and predict system performance. These controllers were then implemented on a physical model using the ESP32 microcontroller.

Experimental results demonstrate that each controller has specific advantages depending on criteria such as settling time, maximum overshoot, and steady-state error.

3 Uvod

Upravljanje nestabilnih i nelinearnih sistema predstavlja značajan izazov u oblasti automatike. Ball and beam sistem je jedan od klasičnih primjera u oblasti automatike i kontrole sistema. Ovaj sistem se često koristi za demonstraciju i testiranje različitih metoda upravljanja zbog svoje inherentne nestabilnosti i nelinearnosti. Ovaj rad se fokusira na projektovanje i implementaciju različitih metoda upravljanja za ball and beam sistem, koristeći ESP32 sistem na čipu.

Glavni cilj rada je istražiti efikasnost različitih pristupa upravljanju, uključujući PID kontroler, fuzzy logički kontroler, te kontroler sa povratnom spregom po cijelom vektoru stanja. U radu će biti prikazan postupak modeliranja sistema, kao i koraci potrebni za realizaciju ovih kontrolera na fizičkom modelu. Posebna pažnja će biti posvećena analizi performansi svakog kontrolera kroz praktična mjerenja.

4 Matematički model

4.1 Lagrangeova metoda

Lagrangeovim postupkom se dobijaju diferencijalne jednačine koje opisuju sistem, ali na takav način da se do tih jednačina dolazi neovisno o referentnom sistemu. Lagrangeova metoda opisuje dinamiku sistema na temelju rada i energije. Jednačine za "ball and beam" sistem dobijamo ovom metodom. Uzete su oznake za parametre i veličine u sistemu:

m - masa lopte

R - poluprečnik lopte

d - poluprečnik zupčanika

J_b - moment inercije lopte

E_k - kinetička energija lopte

E_p - potencijalna energija lopte

α - ugao grede

θ - ugaoni pomjeraj motora

r_{ref} - referentni(željeni) položaj lopte

$r(0)$ - početni položaj lopte

Generalizirane koordinate su date vektorom

$$q(t) = \begin{pmatrix} r(t) \\ \alpha(t) \end{pmatrix}$$

Lagranžijan sistema je dat kao:

$$\mathcal{L} = E_k - E_p \quad (4.1)$$

Kinetička energija lopte je:

$$E_k = \frac{1}{2} J_b \dot{\theta}_b^2 + \frac{1}{2} m v_b^2 \quad (4.2)$$

gdje je $\dot{\theta}_b$ ugaona brzina lopte, a v_b je linearna brzina lopte. Vrijedi da je:

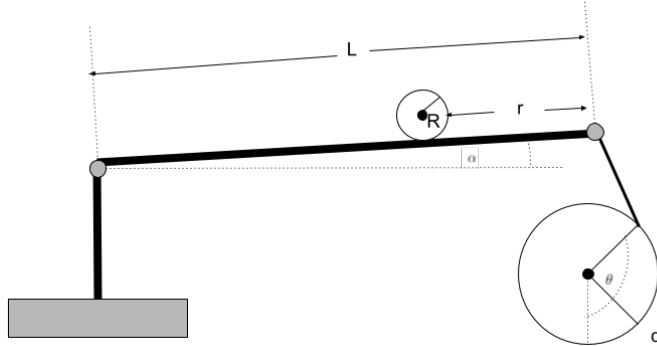
$$\dot{\theta}_b = \frac{\dot{r}}{R} \quad (4.3)$$

Pozicija lopte je izražena kao

$$r_b = \begin{pmatrix} r \cos(\alpha) \\ r \sin(\alpha) \end{pmatrix} = \begin{pmatrix} r_{bx} \\ r_{by} \end{pmatrix} \quad (4.4)$$

Linearnu brzinu lopte možemo izraziti preko generaliziranih koordinata

$$v_b^2 = \dot{r}_{bx}^2 + \dot{r}_{by}^2 = [r \dot{\cos}(\alpha)]^2 + [r \dot{\sin}(\alpha)]^2 = [\dot{r} \cos \alpha - r \dot{\alpha} \sin \alpha]^2 + [\dot{r} \sin \alpha + r \dot{\alpha} \cos \alpha]^2 \quad (4.5)$$



Slika 1: Šema dijagrama sistema lopte i grede

Nakon sređivanja dobija se

$$v_b^2 = \dot{r}^2 + \dot{\alpha}^2 r^2 \quad (4.6)$$

Sada uvrštavanjem (4.6) u (4.1) imamo izraz za kinetičku energiju lopte

$$E_k = \frac{1}{2} J_b \frac{\dot{r}^2}{R^2} + \frac{1}{2} m (\dot{r}^2 + \dot{\alpha}^2 r^2) \quad (4.7)$$

$$E_k = \frac{1}{2} \dot{r}^2 \left(\frac{J_b}{R^2} + m \right) + \frac{1}{2} m \dot{\alpha}^2 r^2 \quad (4.8)$$

Potencijalna energija lopte je:

$$E_p = mgr \sin \alpha \quad (4.9)$$

Sada se može napisati Lagranžijan:

$$\mathcal{L} = E_k - E_p \quad (4.10)$$

$$\mathcal{L} = \frac{1}{2} \dot{r}^2 \left(\frac{J_b}{R^2} + m \right) + \frac{1}{2} m \dot{\alpha}^2 r^2 - mgr \sin \alpha \quad (4.11)$$

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{r}} \right) - \frac{\partial \mathcal{L}}{\partial r} = 0 \quad (4.12)$$

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{r}} \right) = \ddot{r} \left(\frac{J_b}{R^2} + m \right) \quad (4.13)$$

$$\frac{\partial \mathcal{L}}{\partial r} = m \dot{\alpha}^2 r - mg \sin \alpha \quad (4.14)$$

Uvrsti se (4.14) i (4.13) u (4.12) i dobija se

$$\ddot{r} \left(\frac{J_b}{R^2} + m \right) - m \dot{\alpha}^2 r + mg \sin \alpha = 0 \quad (4.15)$$

Ako se izraz (4.15) linearizira za malo $\alpha \rightarrow 0$ ($\sin \alpha \rightarrow \alpha$ i $\cos \alpha \rightarrow 1$) imamo da je:

$$\ddot{r}\left(\frac{J_b}{R^2} + m\right) + mgr \cos \alpha = 0 \quad (4.16)$$

Druga jednačina se dobija na sličan način:

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{\alpha}}\right) = \frac{d}{dt}(mr^2 \dot{\alpha}) = 2mr\dot{r}\dot{\alpha} + mr^2 \ddot{\alpha} \quad (4.17)$$

$$\frac{\partial \mathcal{L}}{\partial \alpha} = -mgr \cos \alpha \quad (4.18)$$

$$2mr\dot{r}\dot{\alpha} + mr^2 \ddot{\alpha} + mgr \cos \alpha = 0 \quad (4.19)$$

Linearizira se jednačina (4.19):

$$mr^2 \ddot{\alpha} + mgr = 0 \quad (4.20)$$

Jednačina koja povezuje ugao grede i ugao zupčanika se može aproksimirati linearnom jednačinom:

$$\alpha = \frac{d}{L}\theta \quad (4.21)$$

Uvrstimo (4.21) u jednačine (4.20) i (4.16) imamo izraze:

$$\ddot{r}\left(\frac{J_b}{R^2} + m\right) + mg\frac{d}{L}\theta = 0 \quad (4.22)$$

$$mr^2 \frac{d}{L}\ddot{\theta} + mgr = 0 \quad (4.23)$$

Jednačine (4.22) i (4.23) predstavljaju linearizirani sistem diferencijalnih jednačina koje opisuju sistem.

4.2 Linearizacija sistema u prostoru stanja

Za vektor stanja uzet je $x(t)$:

$$x(t) = \begin{pmatrix} r \\ \dot{r} \end{pmatrix} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \quad (4.24)$$

$$u(t) = \theta \quad (4.25)$$

$$x_1 = r$$

$$\dot{x}_1 = \dot{r} = x_2$$

$$\dot{x}_2 = \ddot{r}$$

Iz jednačine (4.15) dobijamo da je:

$$\ddot{r} = \frac{-mg \sin \alpha + mr \dot{\alpha}^2}{\frac{J_b}{R^2} + m} \quad (4.26)$$

Za ulaz u sistem uzet će se θ , a kako vrijedi izraz (4.21) onda je:

$$\ddot{r} = \frac{-mg\frac{d}{L}\sin\theta + mr\frac{d}{L}\dot{\theta}^2}{\frac{J_b}{R^2} + m} \quad (4.27)$$

S obzirom da je model koji je korišten takav da imamo zupčanik i polugu koristimo ovaj vektor stanja, a u slučaju da je motor postavljen na sredinu grede onda bi imali vektor stanja koji izgleda ovako $x(t) = \left(r, \dot{r}, \alpha, \dot{\alpha}\right)^T$ i ulaz u sistem bi bio moment sile na gredu. U slučaju koji je obrađen u ovom radu kontroliše se ugaoni pomak motora (koristi se poluga i zupčanik).

Za $\theta \rightarrow 0$ vrijedi $\sin\theta \rightarrow \theta$ i $\dot{\theta} \rightarrow 0$

$$\ddot{r} = \frac{-mg\frac{d}{L}\theta}{\frac{J_b}{R^2} + m} \quad (4.28)$$

Sada jednačinu prostora stanja možemo zapisati kao:

$$\begin{pmatrix} \dot{r} \\ \ddot{r} \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} r \\ \dot{r} \end{pmatrix} + \begin{pmatrix} 0 \\ \frac{-mg\frac{d}{L}}{\frac{J_b}{R^2} + m} \end{pmatrix} \theta \quad (4.29)$$

Izlaz je zadat kao:

$$y = \begin{pmatrix} 1 & 0 \end{pmatrix} \begin{pmatrix} r \\ \dot{r} \end{pmatrix} \quad (4.30)$$

Prenosna funkcija sistema

Iz jednačine (4.28) primjenom Laplasove transformacije dobija se:

$$s^2 R(s) = \frac{-mg\frac{d}{L}}{\frac{J_b}{R^2} + m} \theta(s) \quad (4.31)$$

Prenosnu funkciju sistema sada možemo dobiti kao

$$G(s) = \frac{R(s)}{\theta(s)} = \frac{\frac{-mg\frac{d}{L}}{\frac{J_b}{R^2} + m}}{s^2} = \frac{-mgd}{L(\frac{J_b}{R^2} + m)s^2} \quad (4.32)$$

Može se primjetiti u prenosnoj funkciji da imamo dvostruki pol u nuli što znači da je sistem nestabilan. Bez dodatnog upravljačkog sistema ili povratne veze koja bi stabilizovala loptu, sistem će uvek biti nestabilan, jer lopta neće ostati u ravnoteži na gredi bez obzira na početni uslov. Kada lopta počne da se kreće u jednom smjeru, obzirom da nema stabilizirajućeg faktora, lopta će nastaviti da se kreće dalje od ravnotežnog položaja, umjesto da se vraća u njega. Ovo ponašanje je direktna posljedica nestabilnosti sistema.

5 Kontroleri

Parametri realiziranog modela imaju sljedeće mjerene vrijednosti:

1. Masa lopte (m): 0.2 kg
2. Poluprecnik lopte (R): 0.02 m
3. Gravitaciono ubrzanje (g): -9.8 m/s²
4. Moment inercije lopte (J): 32×10^{-6} kg·m²
5. Poluprecnik zupčanika (d): 3 cm
6. Dužina grede (L): 40 m

Ove vrijednosti parametara sistema će se koristiti za sve kontrolere. Ako zadamo zahtjeve da sistem ima:

$$\begin{aligned} \text{maksimalni preskok } MPOS &= 30\% \\ \text{vrijeme smirenja } T_s &= 2s \end{aligned}$$

Maksimalni preskok i koeficijent prigušenja ζ su povezani jednačinom:

$$\begin{aligned} MPOS &= 100 \times e^{-\zeta\pi/\sqrt{1-\zeta^2}} \\ 0.3 &= e^{-\zeta\pi/\sqrt{1-\zeta^2}} \\ \ln(0.3) &= -\zeta\pi/\sqrt{1-\zeta^2} \\ \zeta &= \frac{-\ln(0.3)}{\pi/\sqrt{1-\zeta^2}} \end{aligned}$$

Ako uvrstimo vrijednosti dobijamo da je ζ :

$$\zeta \approx 0.358$$

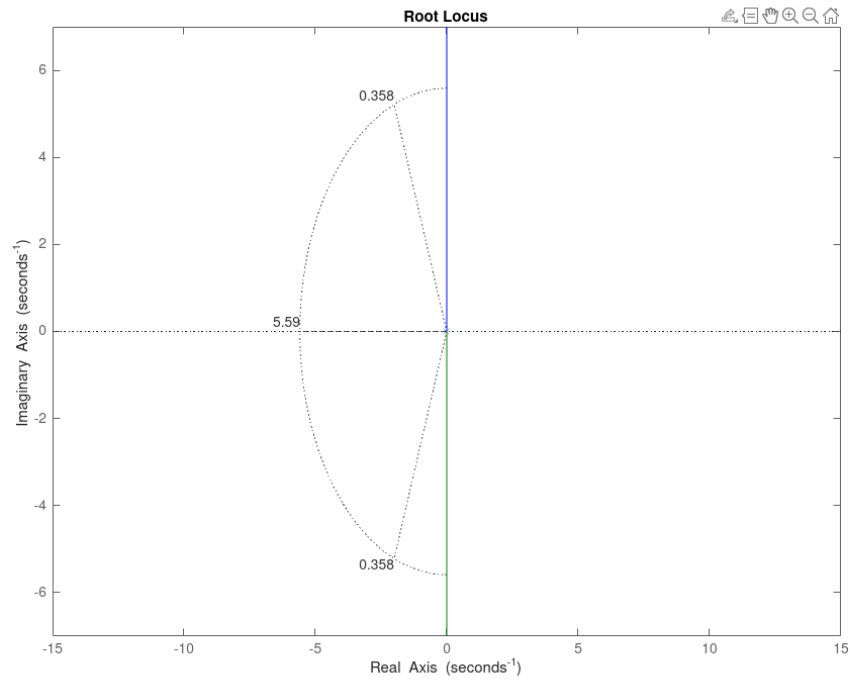
Veza prirodne frekvencije ω_n sa vremenom smirenja je data sa:

$$\begin{aligned} T_s &= \frac{4}{\zeta\omega_n} \\ \omega_n &= \frac{4}{\zeta \cdot T_s} \\ \omega_n &= \frac{4}{0.358 \times 5} \end{aligned}$$

Dobijamo da je ω_n :

$$\omega_n \approx 5.589 \text{ rad/s}$$

Sada možemo nacrtati GMK sistema i prikazati ove zahtjeve.



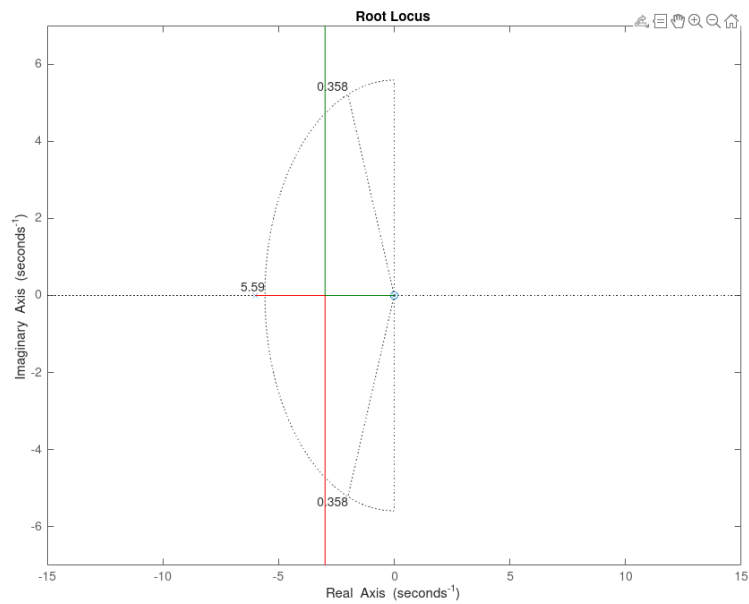
Slika 2: GMK sistema i prostor koji zadovoljava zahtjeve preskoka i vremena smirenja

5.1 Kompenzator sa faznim prednjačenjem

Kompenzator prvog reda sa faznim prednjačenjem pomjera GMK u lijevu poluravan. Kompenzator je dat u formi:

$$C(s) = K_s \frac{(s + z_0)}{s + p_0} \quad (5.1)$$

gdje je z_0 manje amplitude od p_0 . Dodajmo nulu u sistem blizu koordinatnog početka kako bi se poništio efekat jedanog od polova sistema $z_0 = 0.01$. Pol koji se dodaje, dodaje se u lijevoj poluravni kako bi se gmk pomjerio još više u lijevu poluravan. Postavimo pol u $p_0 = 7$. Sada imamo izgled gmk kao na slici:



Slika 3: GMK sistema sa dodatim kompenzatorom, $z_0 = 0.01$ $p_0 = 7$

Ako se izaberu polovi kao na slici 4, u matlabu je omogućeno odrediti potrebna pojačanja koja će omogućiti proizvoljno smještanje polova. To pojačanje dobijamo pozivom funkcije:

```
1 [k, poles]=rlocfind(C*G)
```

pojačanje koje je potrebno za smještanje polova u

$$-3.485 + 2.6874i$$

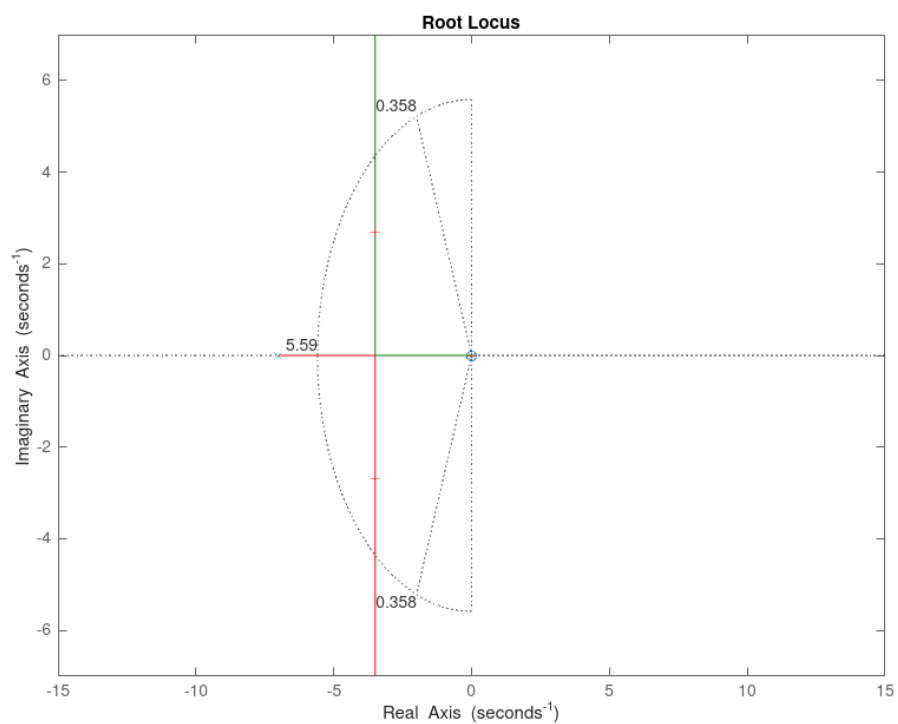
$$-3.485 - 2.6874i$$

$$-0.01$$

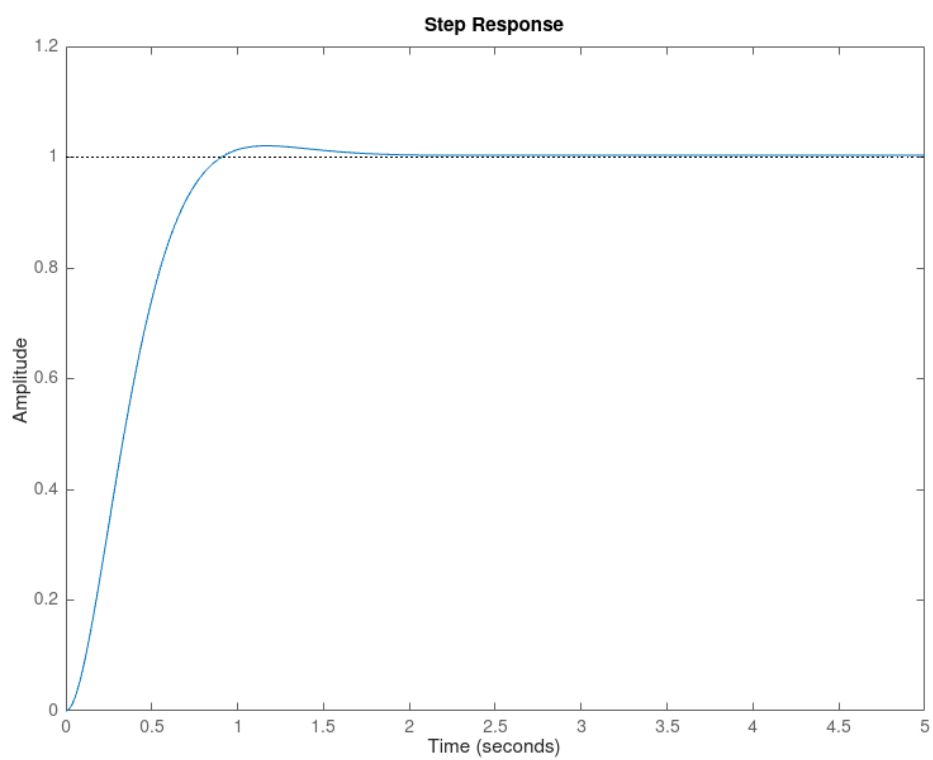
je

$$k = 37.1570$$

Sada možemo vidjeti kakav je odziv sistema sa zatvorenom povratnom spregom na step ulaz, upotrebom matlab funkcije imamo odziv dat kao na slici 5. Sa slike 5 možemo vidjeti da i vrijeme smirenja i maksimalni preskok zadovoljavaju zadate kriterije.



Slika 4: Križićima označeni polovi na GMK



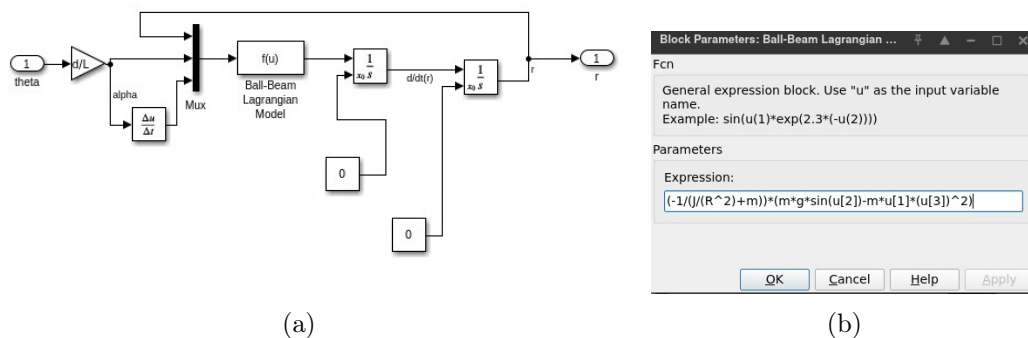
Slika 5: Odziv na step sistema sa kompenzatorom $C(s)$

5.1.1 Matlab kod

```
1 m = 0.2;
2 R = 0.02;
3 g = -9.8;
4 J = 32e-6;
5 d = 3;
6 L = 40;
7 s = tf('s');
8 G = -m*g*d/L/(J/R^2+m)/s^2;
9
10 zo = 0.01;
11 po = 7;
12 C=tf([1 zo],[1 po]);
13
14 rlocus(C*G)
15 sgrid(0.358, 5.589)
16 axis([-15 15 -7 7])
17 k = 37.1570;
18 sys_cl=feedback(k*C*G, 1);
19 t=0:0.01:5;
20 figure
21 step(sys_cl,t)
22 %[k,poles]=rlocfind(C*G)
```

5.1.2 Simulink

Nelinearni model sistema može se predstaviti u simulinku na način kao na slici 6.

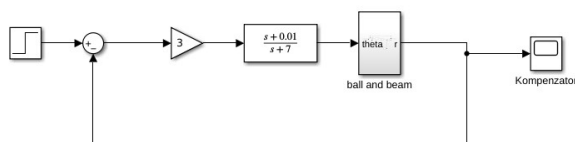


Name	Value	Size	Class
J	3.2000e-05	1x1	double
L	40	1x1	double
R	0.0200	1x1	double
d	3	1x1	double
g	-9.8000	1x1	double
m	0.2000	1x1	double
r	1	1x1	double

(c) Vrijednost parametara u workspace-u

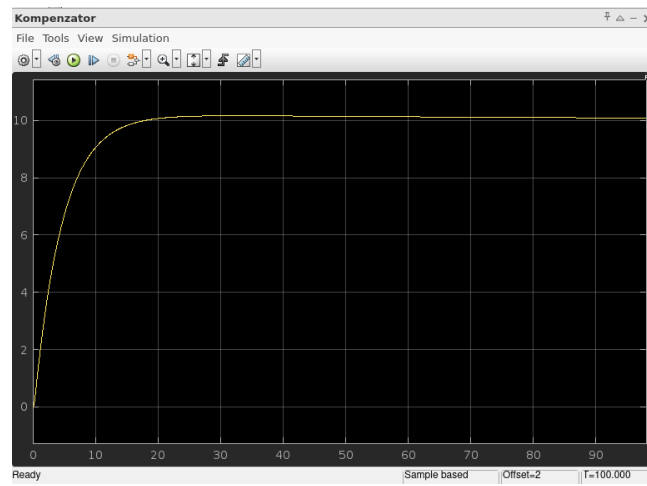
Slika 6: Model sistema u simulink-u

Ako sada primjenimo komenzator prethodno opisan imat cemo blok dijagram kao na slici 7. gdje je blok "ball and beam" grupisani blok elemenata sa slike 6 a).



Slika 7: Model sistema sa dodanim komenzatorom

Pojacanje je znatno manje nego u simulaciji sa linearizovanim modelom. Ako se uzme preveliko pojačanje nastaje problem nestabilnosti i nemogućnost konvergencije odziva sto je jednim dijelom izazvano i numeričkom nestabilnošću. Sada mozemo vidjeti da je odziv sistema kao na slici 8 gdje je vrijeme smirenja približno 10 sekundi, a preskoka nema.



Slika 8: Odziv sistema sa kompenzatorom sa faznim prednjačenjem

5.2 Kontroler sa povratnom spregom po cijelom vektoru stanja

Jednačina prostora stanja je data na sljedeći način:

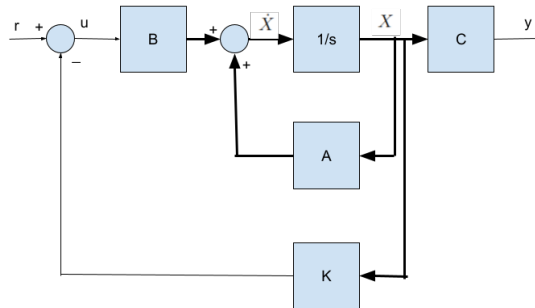
$$\begin{pmatrix} \dot{r} \\ \ddot{r} \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} r \\ \dot{r} \end{pmatrix} + \begin{pmatrix} 0 \\ \frac{-mg\frac{d}{L}}{\frac{J_b}{R^2} + m} \end{pmatrix} \theta \quad (5.2)$$

$$\dot{X} = AX(t) + BU(t)$$

Izlaz je zadat kao:

$$y = \begin{pmatrix} 1 & 0 \end{pmatrix} \begin{pmatrix} r \\ \dot{r} \end{pmatrix} \quad (5.3)$$

$$Y = CX(t)$$



Slika 9: Šema sistema sa povrtanom spregom po vektoru stanja

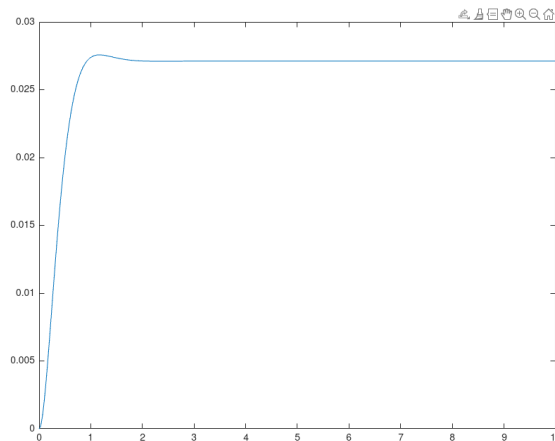
U vektoru stanja je pomjeraj lopte i brzina lopte, a ulaz je ugaoni pomak motora. Karakteristična jednačina sistema sa zatvorenom povratnom spregom je data sa $sI - (A - BK)$. Kako su dimenzije matrica A i BK 2x2, to znaci da želimo definisati položaj 2 pola kako bi se postigao zadovoljavajući odziv. Na osnovu analize GMK iz prethodnog poglavlja, možemo za zeljene polove uzeti:

$$p1 = -3.485 + 2.6874i$$

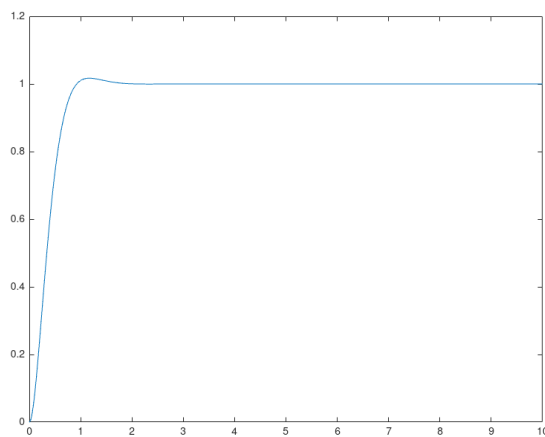
$$p2 = -3.485 - 2.6874i$$

5.2.1 Matlab kod i analiza rezultata

```
1      m = 0.2;
2      R = 0.02;
3      g = -9.8;
4      J = 32e-6;
5      d = 3;
6      L = 40;
7
8      H = -m*g*d/L/(J/(R^2)+m);
9
10     A = [0 1;
11          0 0];
12     B = [0; -m*g*0.075/(J/(R*R)+m)];
13     C = [1 0];
14     D = 0;
15
16     ball_ss = ss(A,B,C,D);
17
18     p1 = -3.485 + 2.6874i;
19     p2 = -3.485 - 2.6874i;
20
21     K = place(A,B,[p1,p2])
22
23     t = 0:0.01:10;
24     u = ones(size(t));
25     sys_cl = ss(A-B*K,B,C,D);
26     [y,t,x] = lsim(sys_cl,u,t);
27     plot(t,y)
```



Slika 10: Odziv sistema sa kontrolerom sa povratnom spregom po vektoru stanja



Slika 11: Odziv sistema sa kontrolerom sa povratnom spregom po vektoru stanja sa uklonjenom greškom u stacionarnom stanju

Primjetno je da za dobijena pojačanja u sistemu sa zatvorenom povratnom spregom imamo veliku grešku u stacionarnom stanju ali je vrijeme smirenja i preskok zadovoljen. Kako bi se izbjegla greška u stacionarnom stanju dodaje se pojačanje reference na takav način da se proračuna željeno stacionarno stanje varijabli stanja, ta vrijednost se pomnoži pojačanjem K i onda se dobijena vrijednost dovodi kao referentna vrijednost.

```

1  s = size(A,1);
2  Z = [zeros([1,s]) 1];
3  N = inv([A,B;C,D])*Z';
4  Nx = N(1:s); Nu = N(1+s);
5  Nbar=Nu + K*Nx;

```

Sada imamo odziv kao na slici 11, i primjetno je da je uklonjena greška u stacionarnom stanju.

5.2.2 Dodavanje integratora u sistem sa povratnom spregom stanja

Da bi se eliminisala greška u stacionarnom stanju u sistemu sa povratnom spregom stanja, potrebno je proširiti naš model stanja tako da uključuje i stanje integratora. Ovo stanje će predstavljati integral greške između željenog izlaza (referentne vrijednosti) i stvarnog izlaza sistema.

Proširenje prostora stanja

Prvo, uvede se nova promjenljiva stanja koja predstavlja integral greške pozicije:

$$\hat{x} = \begin{bmatrix} x \\ x_i \end{bmatrix}$$

gdje je:

- x originalni vektor stanja.
- x_i stanje integratora.

Modifikacija matrica

Sada se trebaju modifikovati matrice A , B i C da bi uključili stanje integratora:

- Nova matrica A :

$$\hat{A} = \begin{bmatrix} A & 0 \\ -C & 0 \end{bmatrix}$$

- Nova matrica B :

$$\hat{B} = \begin{bmatrix} B \\ 0 \end{bmatrix}$$

- Nova matrica C :

$$\hat{C} = \begin{bmatrix} C & 0 \end{bmatrix}$$

5.2.3 Izračunavanje povratne sprege stanja K

Sada možemo izračunati matricu povratne sprege K za prošireni sistem koristeći sledeći MATLAB kod:

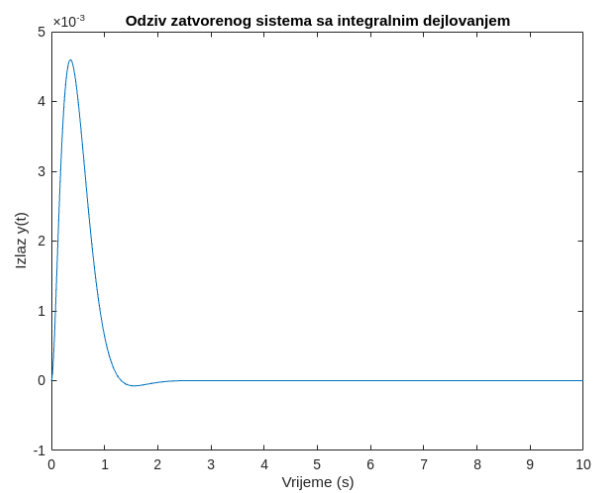
```
1 % Originalni parametri sistema
2 m = 0.2;
3 R = 0.02;
4 g = -9.8;
5 J = 32e-6;
6 d = 3;
7 L = 40;
8
9 H = -m*g*d/L/(J/(R^2)+m);
10
11 A = [0 1;
12      0 0];
```

```

13 B = [0; -m*g*0.075/(J/(R*R)+m)];
14 C = [1 0];
15 D = 0;
16
17 % Prosirene matrice sistema
18 A_aug = [A, zeros(2, 1); -C, 0];
19 B_aug = [B; 0];
20 C_aug = [C, 0];
21
22 % Zeljeni polovi (ukljucujuci jos jedan pol za stanje integratora)
23 p1 = -3.485 + 2.6874i;
24 p2 = -3.485 - 2.6874i;
25 p3 = -10;
26
27 % Izracunavanje povratne sprege stanja za prosireni sistem
28 K_aug = place(A_aug, B_aug, [p1, p2, p3]);
29
30 % Ekstrakcija povratnih koeficijenata
31 K = K_aug(1:2);
32 Ki = K_aug(3);
33
34 % Zatvoreni sistem sa integralnim djelovanjem
35 sys_cl = ss(A_aug - B_aug*K_aug, B_aug, C_aug, D);
36
37 % Simulacija odziva
38 t = 0:0.01:10;
39 u = ones(size(t));
40 [y, t, x] = lsim(sys_cl, u, t);
41 plot(t, y);
42 xlabel('Vrijeme (s)');
43 ylabel('Izlaz y(t)');
44 title('Odziv zatvorenog sistema sa integralnim dejlovanjem');

```

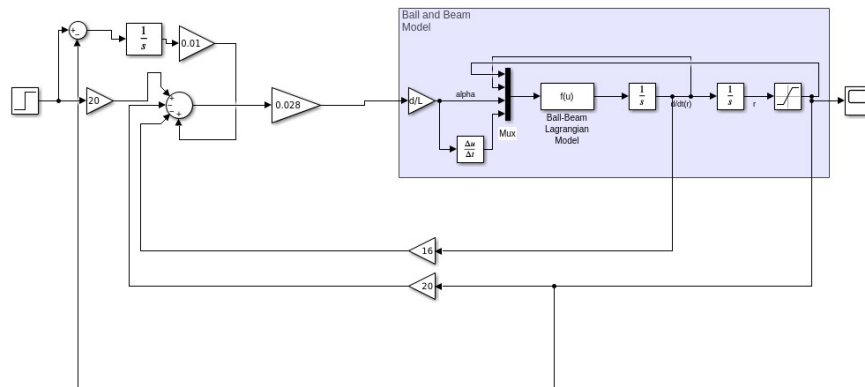
Prva dva elementa vektora K_{aug} odgovaraju povratnim koeficijentima za originalna stanja sistema. Treći element vektora K_{aug} odgovara povratnom koeficijentu za stanje integratora. Simulacija sa uključenim integratorom treba da eliminiše grešku u stacionarnom stanju u odzivu sistema. Odziv izgleda kao na sljedecoj slici.



Slika 12: Odziv sistema sa kontrolerom sa povratnom spregom po vektoru stanja uz dodani integrator u matlabu za linearizirani model

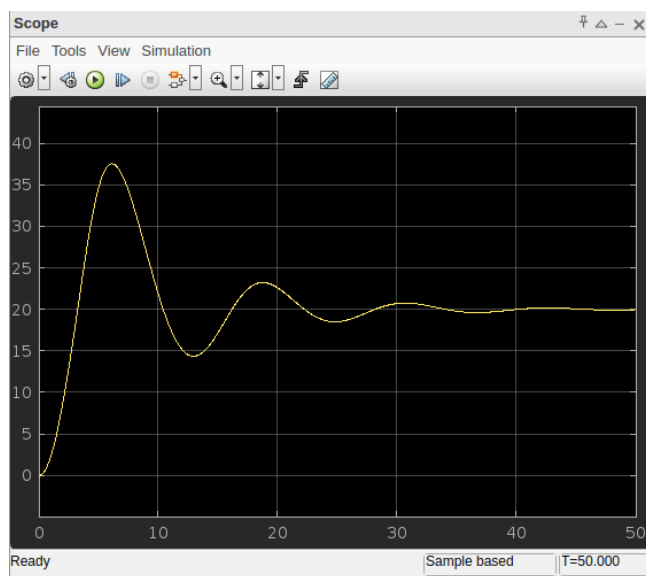
5.2.4 Simulink

Primjenjen kontroler na nelinearni model će u simulinku izgledati kao na slici 13.



Slika 13: Nelinearni model sistema u simulinku sa povratnom vezom po vektoru stanja

Vrijednosti pojačanja su izabrane eksperimentalnim načinom. Pojačanja dobijena u prethodnoj analizi sa lineariziranim modelom su bila prevelika. Sada odziv ima izgled kao na slici 14.



Slika 14: Odziv nelinearnog modela sistema u Simulink-u sa povratnom vezom po vektoru stanja i dodanim integratorom greške pozicije

5.2.5 Implementacija i analiza kontrolera sa povratnom spregom po cijelom vektoru stanja na realnom sistemu

Unaprijedna kontrola

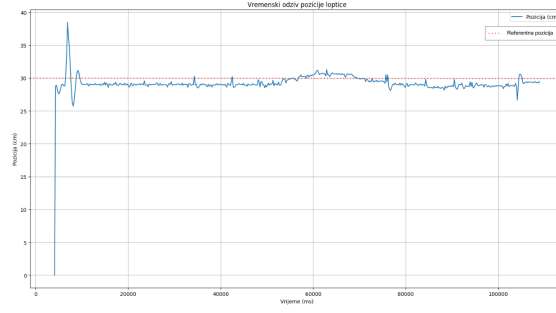
Za realni sistem željena pozicija lopte prvo je postavljena na $r_{ref} = 30$. Za pojačanje $K = [36 \ 13]$ dobija se odziv sistema kao na slici 15(a). Ali ako se pojačanje po poziciji poveća na 38, a po brzini poveća na 16 odziv ima izgled kao na slici 15(b). Primjetimo da je u prvom slučaju vrijeme smirenja približno 5 sekundi a u drugom slučaju sa povećanim pojačanjem vrijeme smirenja je manje od 6 sekundi, a da je maksimalni preskok približno 26% za pojačanja $K = [36 \ 13]$ dok je za pojačanje $K = [38 \ 16]$ preskok 33%. Mapiranje se vrši tako da se proračuna maksimalna i minimalna udaljenost lopte od željene pozicije. U slučaju da je željena pozicija $r_{ref} = 30$, onda je najmanja udaljenost -10, a najveća 30. Sada kako znamo da će se u povratnoj sprezi trenutna vrijednost pozicije pomnožiti pojačanjem K_1 , gdje je $K = [K_1 \ K_2]$, onda da bi izbjegli grešku stacionalnog stanja, referentnu vrijednost pomnožimo istim pojačanjem K_1 . Sada znamo da kontrolni, to jeste ulazni signal je u granicama:

$$(r_{ref} - 40)K_1 < u < (r_{ref} - 0)K_1$$

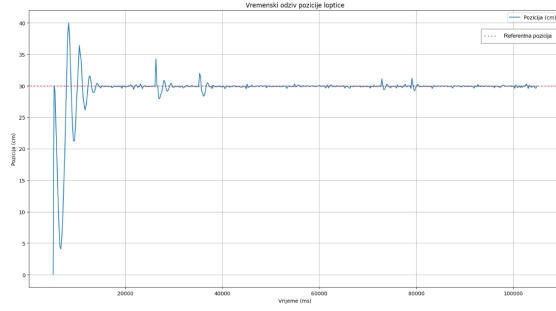
Potrebno je mapirati taj ulaz u ugaoni pomjeraj motora koji ide u granicama od 0 do 180. Kako je ugaoni pomjeraj motora 90 kada je greda paralelna sa zemljom, i $u = 600$ odgovara poziciji od 30 cm na gredi onda je mapiranje odrađeno na sljedeći način:

$$90 < \theta < 180, \quad \text{za } 0 < u \leq 600$$

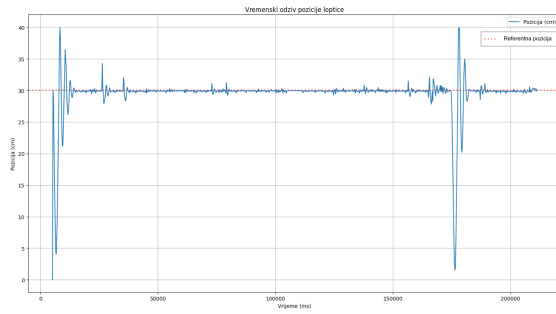
$$0 < \theta < 180, \quad 600 < u < 1200$$



(a) $r_{ref} = 30$ i $K = (36 \ 13)$



(b) $r_{ref} = 30$ i $K = (38 \ 16)$



(c) $r_{ref} = 30$ $K = (38 \ 16)$ otklanjanje greške tipa počet-
nih uslova

Slika 15: Odvaz sistema sa povratnom spregom po vektoru stanja

5.2.6 Unaprijedna kontrola: kod za ESP32

```
1 // Ukljucivanje biblioteke za kontrolu servo motora
2 #include <ESP32Servo.h>
3 // Ukljucivanje WiFi biblioteke
4 #include <WiFi.h>
5 // Ukljucivanje biblioteke za HTTP komunikaciju
6 #include <HTTPClient.h>
7 // Ukljucivanje biblioteke za rad sa JSON formatom
8 #include <ArduinoJson.h>
9 #include <vector>
10
11 const int potPin = 34; // Pin za potencijometar
12 const int servoPin = 12; // Pin za servo motor
13 // Varijabla za očitavanje vrijednosti sa potencijometra
14 int potValue = 0;
15 float potLengthCM = 40.0; // Ukupna dužina potencijometra u centimetrima
16 float positionCM = 0; // Trenutna pozicija lopte u centimetrima
17 float velocityCMperS; // Brzina lopte u centimetrima po sekundi
18 double angle = 0.0; // Ugaoni pomjeraj motora
19 // Vektor za cuvanje pozicija lopte tokom odziva
20 std::vector<float> recordedResponse;
21 // Vektor za cuvanje vremena uzorkovanja
22 std::vector<unsigned long> timeData;
23 int sampleNumber = 0; // Broj uzoraka
24
25 const float m = 0.2; // Masa lopte u kilogramima
26 const float R = 0.02; // Poluprecnik lopte u metrima
27 const float g = -9.8; // Gravitaciona konstanta u m/s^2
28 const double J = 0.000032; // Moment inercije
29 int d = 3; // Poluprecnik zupcanika
30 int L = 40; // Dužina grede u centimetrima
31
32 Servo servo; // Kreiranje objekta za kontrolu servo motora
33
34 // Matrice stanja A i B za sistem
35 const float A[2][2] = {
36     {0, 1},
37     {0, 0}
38 };
39
40 const float B[2][1] = {
41     {0},
42     {-m * g * 0.075 / (J / (R * R) + m)}
43 };
44
45 // Matrica pojačanja K
46 const float K[1][2] = {
47     {38, 16}
48 };
49
```

```

50 void setup() {
51     Serial.begin(921600);
52     servo.attach(servoPin);
53     delay(1000);
54
55     // Inicijalna pozicija lopte na gredi
56     positionCM = (potValue / 4095.0) * potLengthCM;
57 }
58
59 void loop() {
60     // Citanje vrijednosti sa potencijometra
61     potValue = analogRead(potPin);
62
63     // Preracunavanje pozicije u centimetre
64     float newPositionCM = (potValue / 4095.0) * potLengthCM;
65
66     // Ako se pozicija nije znacajno promijenila, dodaj uzorak
67     if((positionCM == newPositionCM) || (positionCM < newPositionCM +
68         0.5) || (positionCM > newPositionCM - 0.5) ){
69         sampleNumber++;
70
71         // Ako je prikupljeno vise od 500 uzoraka, posalji podatke
72         if(sampleNumber > 500){
73             sendData(recordedResponse, timeData); // Slanje podataka
74
75             // Resetovanje vektora
76             recordedResponse = std::vector<float>{};
77             timeData = std::vector<unsigned long>{};
78
79             sampleNumber = 0; // Resetovanje broja uzoraka
80         }
81     }
82
83     // Izracunavanje brzine lopte
84     velocityCMperS = (newPositionCM - positionCM)/0.2;
85
86     // Vektor stanja sistema
87     float x[2] = { newPositionCM, velocityCMperS };
88
89     // Snimanje trenutne pozicije lopte
90     recordedResponse.push_back(positionCM);
91
92     // Snimanje vremena uzorka
93     timeData.push_back(millis());
94
95     // Zeljena pozicija lopte u centimetrima
96     float setpoint_cm = 30;
97
98     // Izracunavanje upravljackog signala koristeći kontroler
99     float u = setpoint_cm * K[0][0] - K[0][0] * x[0] - K[0][1] * x[1];

```

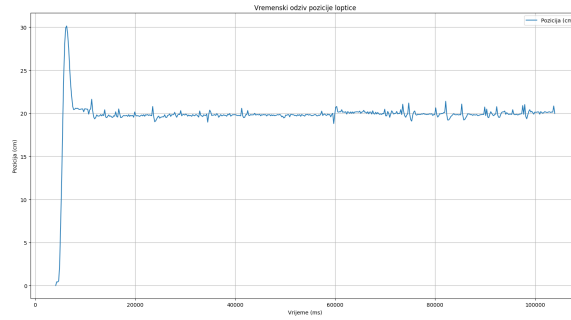
```

100 // Azuriranje položaja motora na osnovu upravljackog signala
101 updateMotor(u, setpoint_cm);
102
103 // Azuriranje trenutne pozicije lopte
104 positionCM = newPositionCM;
105
106 delay(100); // Kasnjenje od 100 ms za stabilizaciju sistema
107 }
108
109 void updateMotor(float controlInput, float setpoint_cm) {
110     const int u_max = (setpoint_cm - 0)*K[0][0]; // Maksimalni
111         upravljacki signal
112     const int u_min = (setpoint_cm - 40)*K[0][0]; // Minimalni
113         upravljacki signal
114     int servoAngle;
115
116     // Mapiranje upravljackog signala na ugao servo motora
117     if(controlInput <= 600){
118         servoAngle = map(controlInput, 0, 600, 90, 180);
119     } else {
120         servoAngle = map(controlInput, 600, 1200, 0, 180);
121     }
122
123     servoAngle = constrain(servoAngle, 0, 180); // Ogranicavanje ugla
124         servo motora na 0-180 stepeni
125     servo.write(servoAngle); // Slanje upravljackog signala servo
126         motoru
127     delay(100); // Kasnjenje od 100 ms za stabilizaciju motora
128 }

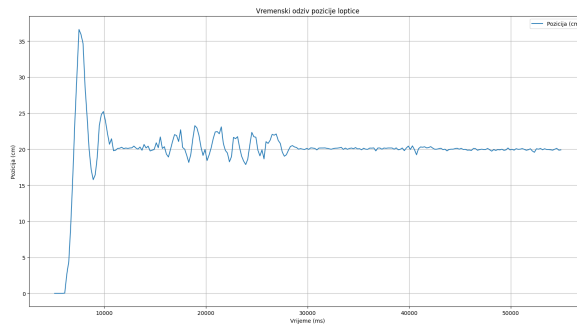
```

Dodavanje integratora u kod za realni sistem i analiza rezultata

Ako se sada za referentnu vrijednost uzme $r_{ref} = 20$ i ako se doda integrator u kodu za realni model onda za pojačanje $K_i = 0.1$, $K = (20, 16)$ imamo odziv kao na slici 16 a). Može se vidjeti da je preskok 50%, a da je vrijeme smirenja približno 7 sekundi. A ako se za pojačanje uzme K približno kao u Simulink simulaciji $K = (20 * 0,03, 16 * 0,03)$ imamo odziv kao na slici 16 b). U tom slučaju i preskok i vrijeme smirenja se poklapaju sa rezultatima u Simulink simulaciji.



(a) $K_i = 0.1$, $K = (20, 16)$



(b) $K_i = 0.02$, $K = (20 * 0.03, 16 * 0.03)$

Slika 16: Odziv sistema sa kontrolerom sa povratnom spregom vektora stanja uz dodani integrator $r_{ref} = 20$

U kodu su dodane definicije varijabli:

```
1 double Ki = 0.02;
2 double cumulativeError = 0;
```

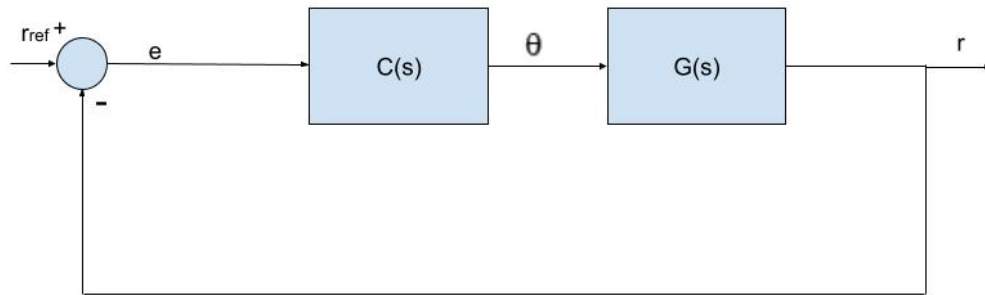
Također je promjenjen i upravljački signal u glavnoj petlji tako da ima sljedeći oblik:

```
1 float x[2] = { newPositionCM, velocityCMperS };
2 float setpoint_cm = 20;
3 float error = setpoint_cm - x[0];
4 cumulativeError += error;
5 float u = (error)*K[0][0] - K[0][1] * x[1] + cumulativeError*Ki;
```

Zadržano je unaprijedno upravljanje, ali je dodan i integrator.

5.3 PID kontroler

Prenosna funkcija sistema u otvorenoj povratnoj sprezi data je sa jednačinom (4.32). Sistem sa zatvorenim povratnom spregom izgleda kao na slici 17.



Slika 17: Blok dijagram sistema zadatog prenosnom funkcijom $G(s)$ sa kontrolerom $C(s)$

Prenosna funkcija PID kontrolera ima oblik:

$$C(s) = K_p + \frac{K_i}{s} + K_d s = \frac{K_d s^2 + K_p s + K_i}{s} \quad (5.4)$$

5.3.1 Matlab kod i analiza rezultata

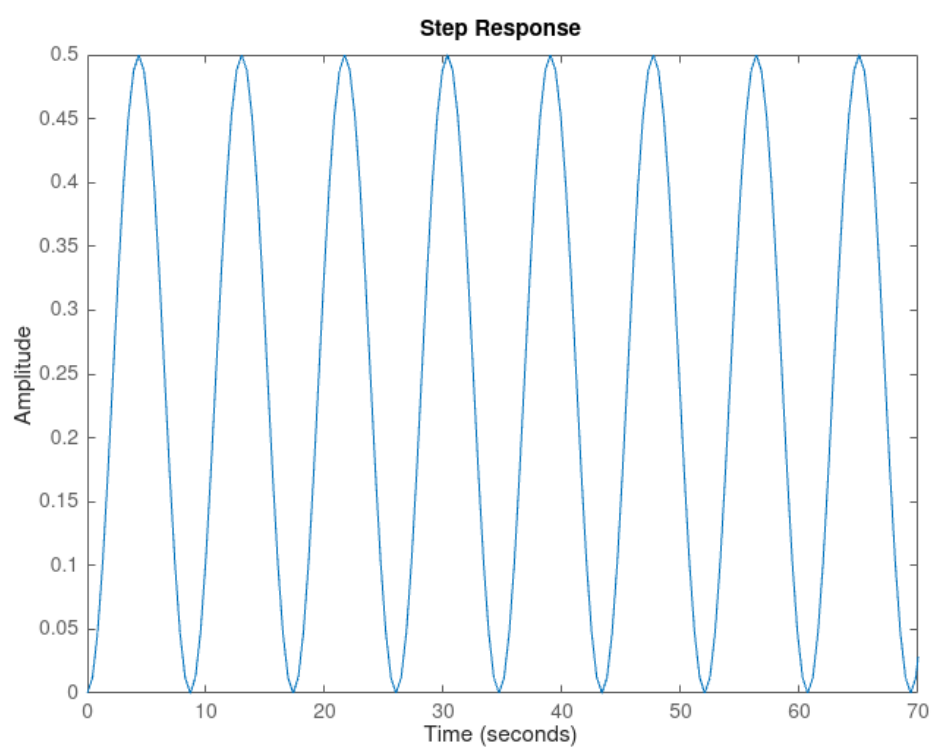
Neka se prvo postavi proporcionalno pojačanje K_p i analizirat će se vremenski odziv. Matlab kod je sljedeći:

```
1 m = 0.2;
2 R = 0.02;
3 g = -9.8;
4 J = 32e-6;
5 d = 3;
6 L = 40;
7
8 s = tf('s');
9 G = -m*g*d/L/(J/R^2+m)/s^2;
10
11 Kp = 1;
12
13 C = pid(Kp);
14
15 sys_cl=feedback(C*G,1);
16
17 step(0.25*sys_cl)
18 axis([0 70 0 0.5])
19 info = stepinfo(sys_cl);
20
21 disp(['Preskok: ', num2str(info.Overshoot), '%']);
22 disp(['Vrijeme smirenja: ', num2str(info.SettlingTime), ' seconds']);
23
24 poles = pole(sys_cl);
25 disp('Polovi zatvorenog sistema:');
26 disp(poles);
27
28 zeros_sys = zero(sys_cl);
29 disp('Nule zatvorenog sistema:');
30 disp(zeros_sys);
```

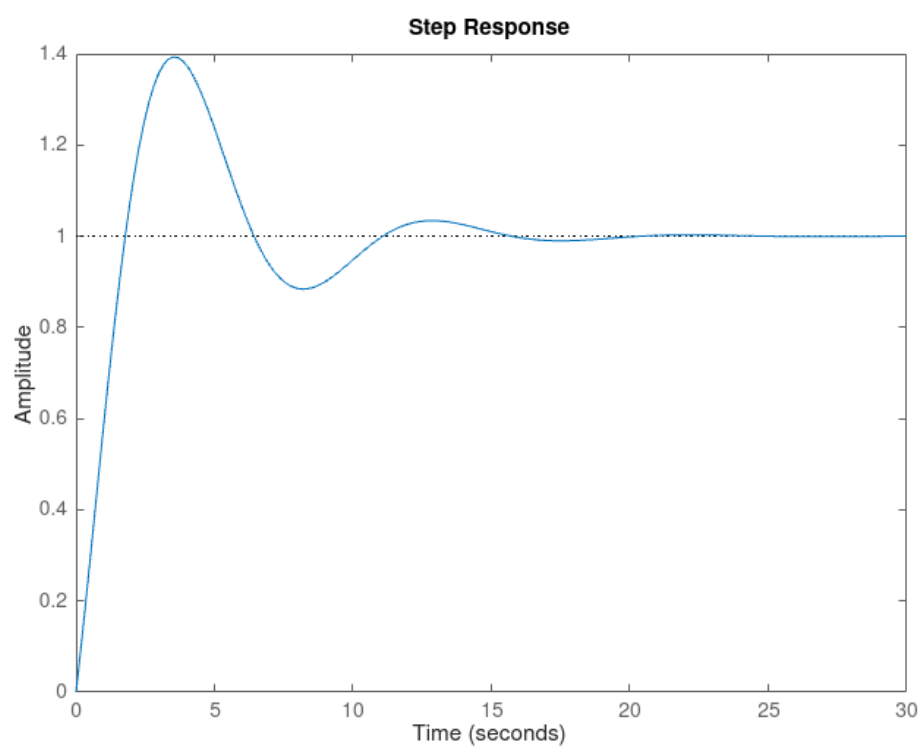
Kao što se vidi odziv sistema oscilira i nema smirenja (slika 18.). Sistem je idalje nestabilan. Ako sada dodamo derivacionu komponentu to jeste pojačanje K_d imamo sljedeći kod

```
1
2 Kp = 1;
3 Kd = 1;
4
5 C = pid(Kp,0,Kd);
6
7 sys_cl=feedback(C*G,1);
8
9 step(sys_cl)
```

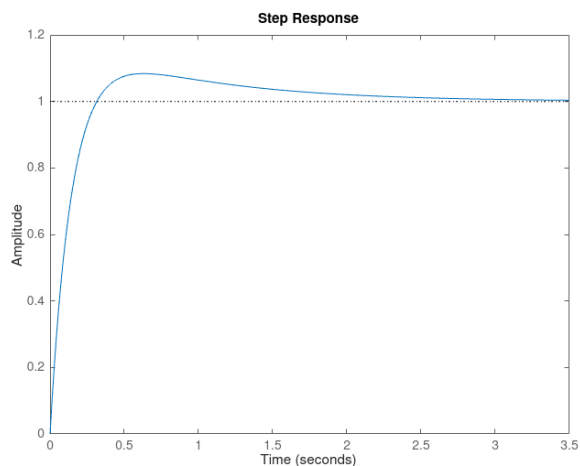
Sada je odziv sistema kao na slici 19. Vrijeme smirenja je 14.3456 sekundi i maksimalni preskok je 39,31%. Vidimo da je derivaciona komponenta popravila i vrijeme smirenja i preskok. Ako se želi postići još manje vrijeme smirenja to se postiže povećanjem pojačanja K_d i K_p . Imamo sljedeće vrijednosti:



Slika 18: Odziv sistema sa PID kontrolerom sa samo sliciproporcionalnom komponentom



Slika 19: Odziv sistema kontrolisan PID kontrolerom sa proporcionalnom i derivacionom komponentom



Slika 20: Odziv sistema kontrolisan PID kontrolerom za $K_p = 15$ i $K_d = 15$

```

1 Kp = 15;
2 Kd = 15;
3 C = pid(Kp,0,Kd);

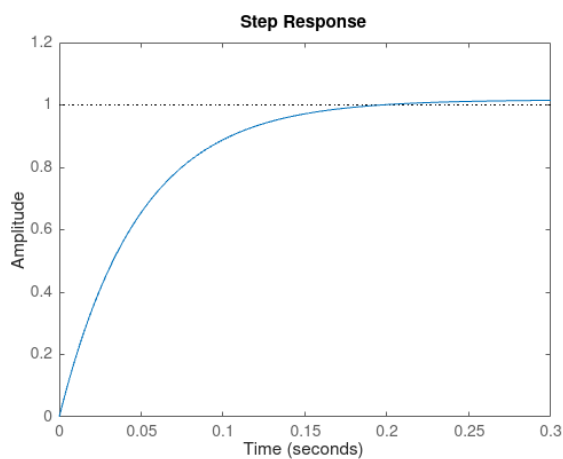
```

Na slici 20 prikazan je odziv. Vrijeme smirenja se smanjilo na 2 sekunde i preskok je 8%. Ako se pojačanja postave na sljedeći način imamo odziv kao na slici 21.

```

1 Kp = 15;
2 Kd = 40;
3 Ki = 0.2;

```

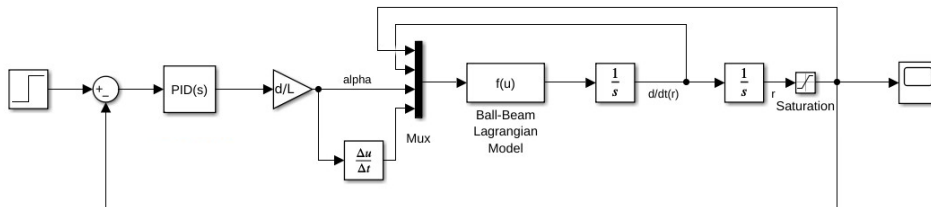


Slika 21: Odziv sistema sa PID kontrolerom $K_p = 15$ $K_d = 40$ $K_i = 0.2$

Najbolji rezultati su dobijeni za pojačanja $K_p = 15$ $K_d = 40$ $K_i = 0.2$. Preskok je 1.5935%, vrijeme smirenja 0.15984 sekunde. Polovi zatvorenog sistema su smješteni u -20.6181 i -0.3819. Dok je nula zatvorenog sistema u -0.3750.

5.3.2 Simulink

Primjenjujući PID na nealinarni model sistema dobijamo blok dijagram kao na slici 22.



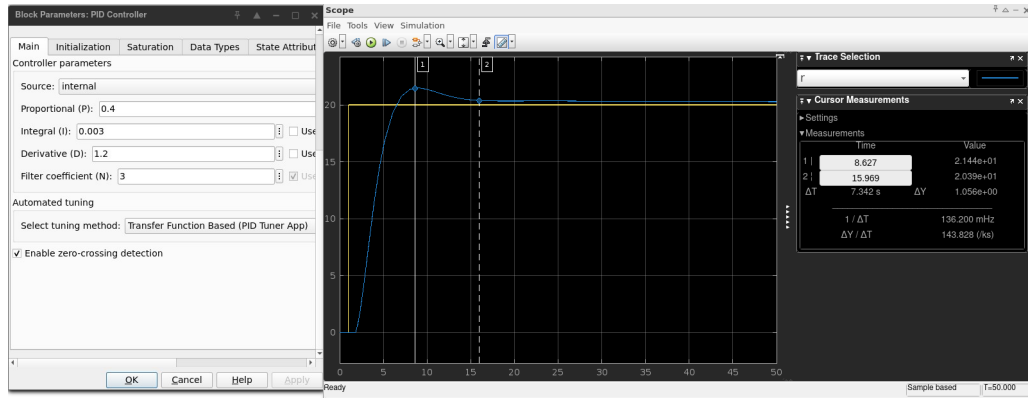
Slika 22: PID kontroler primjenjen na nelinearni model

Sada za različite vrijednosti pojačanja imamo različite odzive (preskoke i vrijeme smirenja) koji će biti prikazani u sljedećoj tabeli gdje je $r_{ref} = 20$.

$K_p \ K_i \ K_d$	T_s , MPOS	Odziv
$K_p = 0.2 \ K_i = 0 \ K_d = 0$	∞ , 100%	
$K_p = 0.4 \ K_i = 0 \ K_d = 0.6$	25s, 80%	
$K_p = 0.2 \ K_i = 0.01 \ K_d = 0.5$	50s, 55%	

Kao što se može primjetiti primjena samo prporcionalnog pojačanja je ponovo rezultovala konstantnim oscilacijama to jeste nestabilnošću. Kada se dodalo derivativno pojačanje popravio

se i preskok i vrijeme smirenja ali su vrijednosti idalje visoke. Dodavanjem pojačanja K_i i smanjenjem procorcionalnog pojačanja K_p preskok se smanjio ali se vrijeme smirenja povećalo. Daljnjim eksperimentalnim namještanjem i iteracijama se dobivaju parametri i odzivi kao na slici 23.



Slika 23: Odziv sistema sa PID kontrolerom $T_s = 15.96$ $MPOS = 7\%$ $K_p = 0.4$ $K_i = 0.003$ $K_d = 1.2$

Preskok je značajno smanjen kao i vrijeme smirenja. Pokušano je prvo imitirati omjere pojačanja K_p i K_d koji su dobijeni u simulaciji lineariziranog modela u poglavlju 5.3.1 i malom modifikacijom proporcionalnog pojačanja dobijen je odziv kao na slici 23.

5.3.3 Implementacija i analiza PID kontrolera na realnom sistemu

Kod za PID kontroler realizovan na ESP32 polčici je sljedeći:

```
1 #include <ESP32Servo.h>
2 #include <WiFi.h>
3 #include <HTTPClient.h>
4 #include <ArduinoJson.h>
5 #include <vector>
6
7 const char* ssid = "";
8 const char* password = "";
9
10 const int potPin = 34;
11 const int servoPin = 12;
12 int potValue = 0;
13 float potLengthCM = 40.0;
14 float positionCM;
15
16 // Prethodni ugao servo motora
17 double previousAngle = 0;
18
19 // Novi ugao servo motora
20 double newAngle = 0;
21
22 // Kumulativna greska za I komponentu PID kontrolera
23 double cumulativeError = 0;
24
25 // Prethodna greska za D komponentu PID kontrolera
26 double previousError = 0;
27
28 std::vector<float> recordedResponse;
29 std::vector<unsigned long> timeData;
30 int sampleNumber = 0;
31
32 double Kp = 15; // Proporcionalno pojačanje za PID kontroler
33 double Ki = 0.2; // Pojačanje integral komponente PID kontrolera
34 double Kd = 40; // Pojačanje dervativne komponente PID kontrolera
35
36
37 Servo servo;
38
39 void setup() {
40     Serial.begin(921600);
41     servo.attach(servoPin);
42     delay(1000);
43
44     WiFi.begin(ssid, password);
45     while (WiFi.status() != WL_CONNECTED) {
46         delay(1000);
47         Serial.println("Connecting to WiFi...");
48     }
```

```

49     Serial.println("Connected to WiFi");
50 }
51
52 void loop() {
53
54     // Citanje vrijednosti sa potencijometra i preračunavanje u centimetre
55     potValue = analogRead(potPin);
56     float newPositionCM = (potValue / 4095.0) * potLengthCM;
57
58     if((positionCM == newPositionCM) || (positionCM < newPositionCM +
59         0.5) || (positionCM < newPositionCM - 0.5) ){
60         sampleNumber++;
61         if(sampleNumber > 500){
62             sendData(recordedResponse, timeData);
63             recordedResponse.clear();
64             timeData.clear();
65             sampleNumber = 0;
66         }
67     }
68
69     // Azuriranje trenutne pozicije lopte
70     positionCM = newPositionCM;
71
72     recordedResponse.push_back(positionCM);
73     timeData.push_back(millis());
74
75     // Izračunavanje novog ugla pomoću PID kontrolera
76     newAngle = pid(positionCM);
77     // Azuriranje položaja motora na osnovu novog ugla
78     updateMotor(newAngle);
79     previousAngle = newAngle;
80     delay(100);
81 }
82
83 void updateMotor(double new_tilt_deg) {
84     servo.write(new_tilt_deg);
85 }
86
87 double pid(double distance_cm) {
88     //Zeljena pozicija lopte u cm
89     double setpoint_cm = 25;
90
91     double error = setpoint_cm - distance_cm;
92     if(error<0.5 && error>-0.5){
93         error = 0; //ako je greska u granicama -0.5 i 0.5 ignorisi je
94     }
95     double p_value = error * Kp;
96
97     double i_value = cumulativeError * Ki;
98

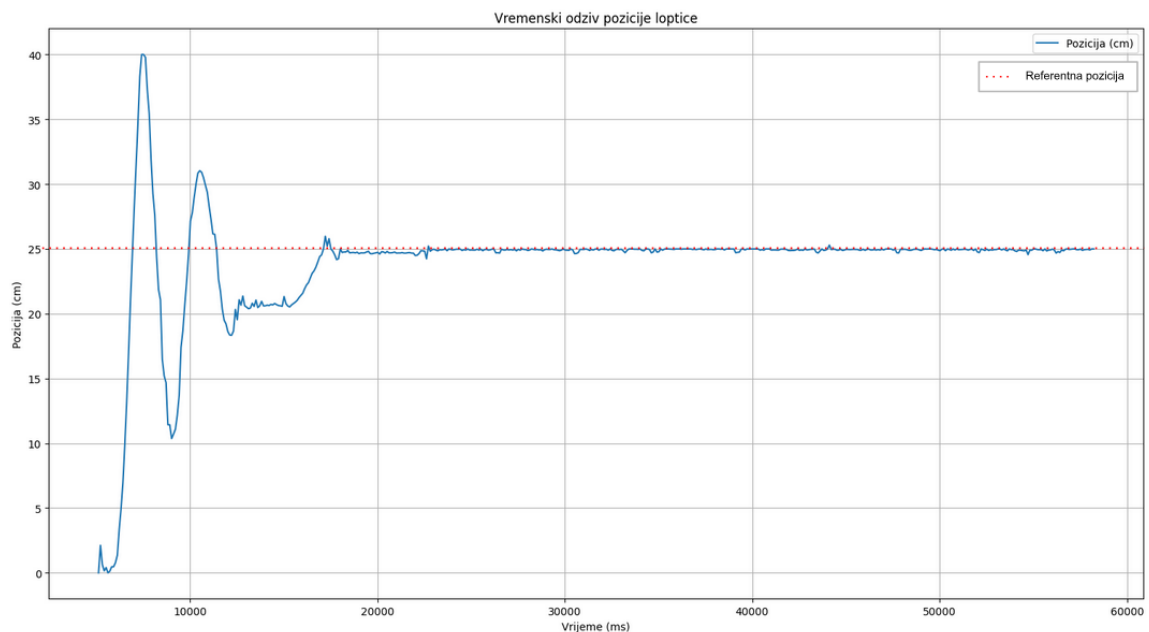
```

```

99     double d_value = (error - previousError) * Kd;
100
101     double pid_value = p_value + i_value + d_value;
102
103     // Gornja i donje granica izlazne vrijednosti PID kontrolera
104     const int upper_limit = (setpoint_cm - 0)*Kp;
105     const int lower_limit = (setpoint_cm - 40)*Kp;
106
107     cumulativeError += error;
108     previousError = error;
109
110     double new_servo_angle = map(pid_value, lower_limit, upper_limit, 0,
111                                  180);
112     // Povratna vrijednost novog ugla za servo motor
113     return new_servo_angle;
114 }

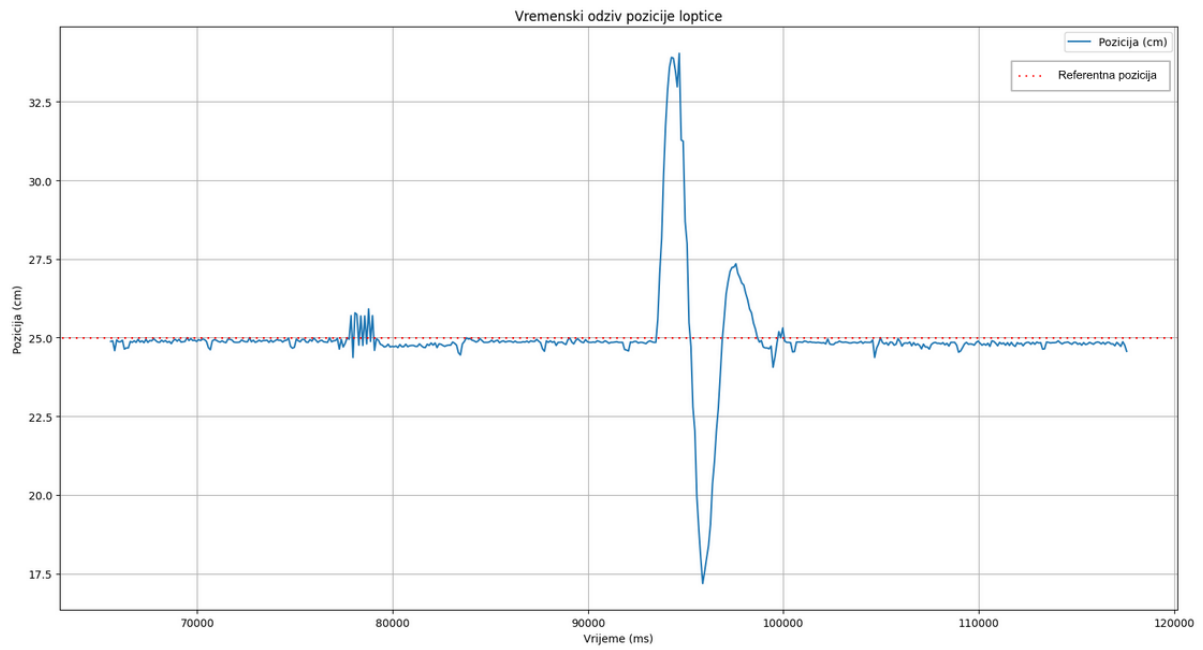
```

Funkcija sendData služi kako bi se na server poslali zabilježeni rezultati mjerenja, pozicija lopte u vremenu. Na tom serveru se rezultati primaju i iscrtava se grafik koji se vidi na slici 24. Ako se na realni sistem primjene pojačanja koja daju najbolje vrijednosti maksimalnog preskoka i vremena smirenja dobijena matlab simulacijom za idealni sistem onda ćemo imati odziv kao na slici 24 za $r_{ref} = 25$.



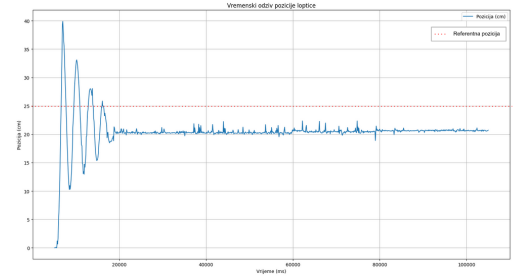
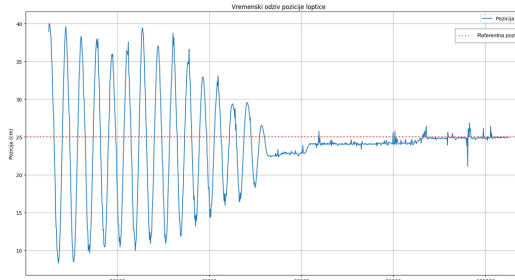
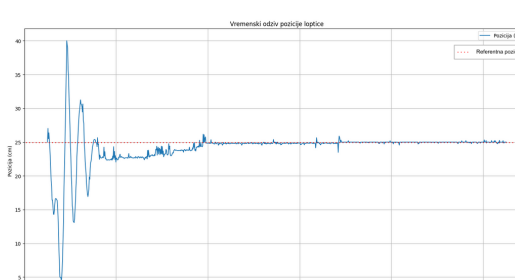
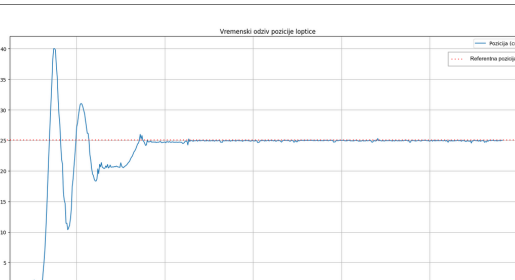
Slika 24: Odziv realnog sistema sa pojačanjima $K_p = 15$ $K_d = 40$ $K_i = 0.2$

Primjetimo da je vrijeme smirenja približno 10 sekundi. Ako je lopta izmještena na sami kraj grede $r(0) = 0$, tada je preskok veoma veliki iznosi oko 60% ako posmatramo od trenutka kada na loptu počne djelovati greda, ali ako posmatran odziv na slici 25 gdje je lopta izmaknuta za manju vrijednost $r(0) = 33$ (u odnosu na referentni položaj) tada će i preskok biti manji približno 32% i vrijeme smirenja je približno 5 sekundi.



Slika 25: Odziv realnog sistema sa PID kontrolerom uz pocetne uslove $r(0) = 33$

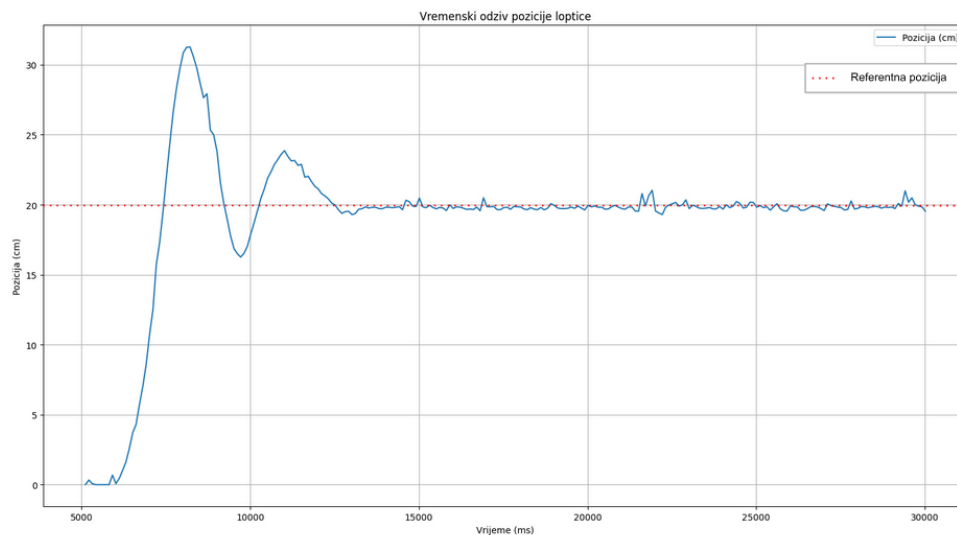
$K_p \ K_i \ K_d$	T_s , MPOS	Odziv
$K_p = 15 \ K_i = 0 \ K_d = 0$	∞ , 100%	
$K_p = 1.5 \ K_i = 0 \ K_d = 1$	20s, 60%	

$K_p = 15 \ K_i = 0 \ K_d = 15$	13s, 60%	
$K_p = 15 \ K_i = 0.1 \ K_d = 15$	50s, 80%	
$K_p = 15 \ K_i = 0.1 \ K_d = 30$	30s, 60%	
$K_p = 15 \ K_i = 0.2 \ K_d = 40$	10s, 60%	

Prilikom proračuna vremena smirenja u obzir se uzima trenutak od kojeg sistem počinje reagovati na ulaz, a to je 5 sekundi nakon pokretanja programa i snimanja podataka. Početni položaj iz kojeg je lopta prepustena sistemu je posljedica ručnog izmještanja lopte.

Za $K_p = 15 \ K_i = 0 \ K_d = 15$ i $K_p = 15 \ K_i = 0.2 \ K_d = 40$ imamo isto vrijeme smirenja i preskok, ali se zato greška u stacionarnom stanju znatno razlikuje. Kada nemamo I komponentu PID regulatora u prva tri slučaja javlja se greška u stacionarnom stanju od približno 5cm. Iako I komponenta u ovakav sistem uvodi nestabilnost ta nestabilnost će se poništiti drugim pojačanjima. I komponenta PID regulatora je bitna radi otklanjanja greške u stacionarnom stanju sistema.

Ako sada primjenimo na realni sistem pojačanja dobijena eksperimentalno u Simulink-u onda imamo odziv kao na slici 26. Vrijeme smirenja je približno 7.5 sekundi a preskok je 55%. To su bolje vrijednosti u poređu sa odzivom sistema na koji su primjenjena pojačanja dobivena simulacijom lineariziranog modela sistema (odziv realnog sistema na koji su primjenjena ta pojačanja dat je na slici 24). Iako se vrijeme smirenja i preskok ne poklapa sa simulacijom u Simulink-u na slici 23. ipak su rezultati mjerenja i na realnom sistemu, za dobijena pojačanja, najbolja.



Slika 26: $K_p = 0.4$ $K_i = 0.003$ $K_d = 1.2$

Jedan od razloga što se odzivi na slici 23. i 26. ne poklapaju je nesavršenost modela i mjerenja parametara sistema, te način na koji se mapira ugao koji se prosljeđuje motoru. Na realnom sistemu je potrebno dodatno mapirati dobijenu vidjenost od PID-a u interval (0, 180) koji se prosljeđuje servo motoru.

Pored pojačanja koja su promjenjena pri posljednjem mjerenju (slika 26) također je i postavljena granica na maksimalnu vrijednost ugla θ što je također poboljšalo odziv, ta promjena je napravljena u funkciji updateMotor na sljedeći način:

```

1 void updateMotor(double new_tilt_deg) {
2     new_tilt_deg = constrain(new_tilt_deg, 30, 100);
3     servo.write(new_tilt_deg);
4 }

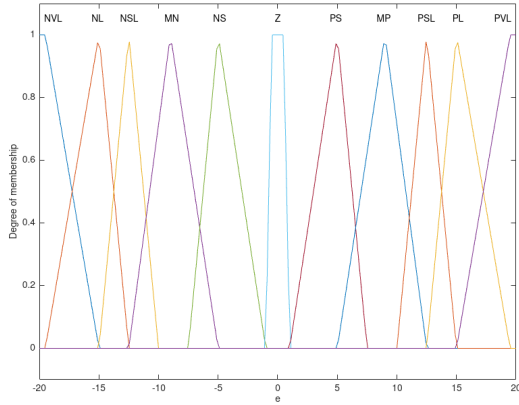
```

5.4 Fuzzy kontroler

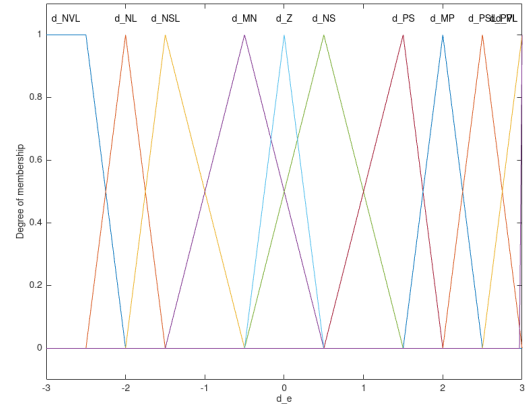
Ball and beam sistem je poznat po svojoj visokoj nelinearnosti i osjetljivosti na promjene u početnim uslovima i spoljnom opterećenju. Fuzzy upravljanje predstavlja metod koji omogućava modeliranje i kontrolu takvih složenih sistema koristeći lingvističke izraze i pravila zasnovana na ljudskom razumijevanju i intuiciji. U fuzzy upravljanju, ulazni i izlazni parametri se predstavljaju u obliku fuzzy varijabli, a pravila se definišu kao skupovi ako-onda izjava koje omogućavaju fleksibilan odgovor na različite situacije.

Ulazi

- **Greška (Error, e):** Ovaj ulaz predstavlja razliku između željenog položaja i trenutnog položaja lopte. Funkcije pripadnosti za grešku su:
 - *NVL* (Very Large Negative): $\text{trapmf}([-20, -20, -19.5, -15])$
 - *NL* (Large Negative): $\text{trimf}([-19.5, -15, -12.5])$
 - *NSL* (Negative Slightly Large): $\text{trimf}([-15, -12.5, -10])$
 - *MN* (Negative Medium): $\text{trimf}([-12.5, -9, -5])$
 - *NS* (Negative Small): $\text{trimf}([-7.5, -5, -1])$
 - *Z* (Zero): $\text{trapmf}([-1, -0.5, 0.5, 1])$
 - *PS* (Positive Small): $\text{trimf}([1, 5, 7.5])$
 - *MP* (Positive Medium): $\text{trimf}([5, 9, 12.5])$
 - *PSL* (Positive Slightly Large): $\text{trimf}([10, 12.5, 15])$
 - *PL* (Positive Large): $\text{trimf}([12.5, 15, 19.5])$
 - *PVL* (Positive Very Large): $\text{trapmf}([15, 19.5, 20, 20])$
- **Promjena greške (DeltaError, d_e):** Ovaj ulaz predstavlja promjenu greške između trenutnog i prethodnog uzorka. Funkcije pripadnosti za promjenu greške su:
 - d_NVL (Very Large Negative): $\text{trapmf}([-3, -3, -2.5, -2])$
 - d_NL (Large Negative): $\text{trimf}([-2.5, -2, -1.5])$
 - d_NSL (Negative Slightly Large): $\text{trimf}([-2, -1.5, -0.5])$
 - d_MN (Negative Medium): $\text{trimf}([-1.5, -0.5, 0.5])$
 - d_NS (Negative Small): $\text{trimf}([-0.5, 0.5, 1.5])$
 - d_Z (Zero): $\text{trapmf}([-0.5, 0, 0, 0.5])$
 - d_PS (Positive Small): $\text{trimf}([0.5, 1.5, 2])$
 - d_MP (Positive Medium): $\text{trimf}([1.5, 2, 2.5])$
 - d_PSL (Positive Slightly Large): $\text{trimf}([2, 2.5, 3])$
 - d_PL (Positive Large): $\text{trimf}([2.5, 3, 3.5])$
 - d_PVL (Positive Very Large): $\text{trapmf}([3, 3, 3.5, 4])$

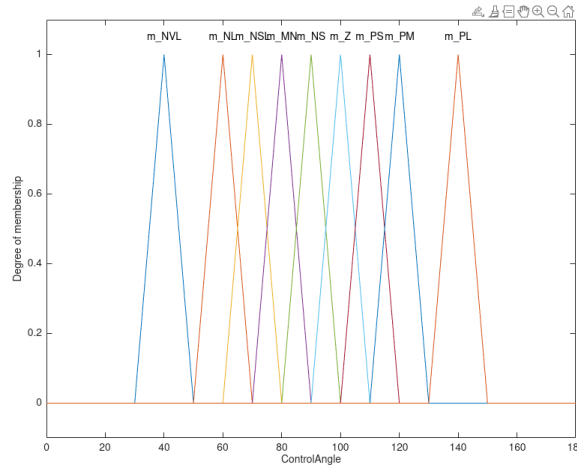


(a) Funkcije pripadnosti greške $e = r - r_{ref}$



(b) Funkcije pripadnosti za promjenu greške

Slika 27: Funkcije pripadnosti ulaza za $r_{ref} = 20$



Slika 28: Funkcije pripadnosti izlaza

Izlazi

- **Teta (θ):** Ovaj izlaz predstavlja ugao servo motora koji se koristi za pomjeranje grede. Funkcije pripadnosti za θ su definisane u sljedećim opsezima:

- m_NVL (Very Large Negative): $\text{trimf}([30, 40, 50])$
- m_NL (Large Negative): $\text{trimf}([50, 60, 70])$
- m_NSL (Negative Slightly Large): $\text{trimf}([60, 70, 80])$
- m_MN (Medium Negative): $\text{trimf}([70, 80, 90])$
- m_NS (Small Negative): $\text{trimf}([80, 90, 100])$
- m_Z (Zero): $\text{trimf}([90, 100, 110])$
- m_PS (Small Positive): $\text{trimf}([100, 110, 120])$
- m_PM (Medium Positive): $\text{trimf}([110, 120, 130])$
- m_PL (Large Positive): $\text{trimf}([130, 140, 150])$

Na slici 28 možemo vidjeti vizuelni prikaz veze ulaza i izlaza fuzzy kontrolera ako su pravila definisana na sljedeći način:

```

1 ruleList = [
2 "e==NVL & d_e==d_PVL => teta=m_NVL";
3 "e==NL & d_e==d_PL => teta=m_NL";
4 "e==NSL & d_e==d_PSL => teta=m_NSL";
5 "e==MN & d_e==d_NS => teta=m_MN";
6 "e==NS & d_e==d_Z => teta=m_NS";
7 "e==Z & d_e==d_PS => teta=m_Z";
8 "e==PS & d_e==d_MP => teta=m_PS";
9 "e==MP & d_e==d_PSL => teta=m_PM";
10 "e==PSL & d_e==d_NVL => teta=m_PL";
11 "e==PL & d_e==d_NL => teta=m_PL";
12 ];

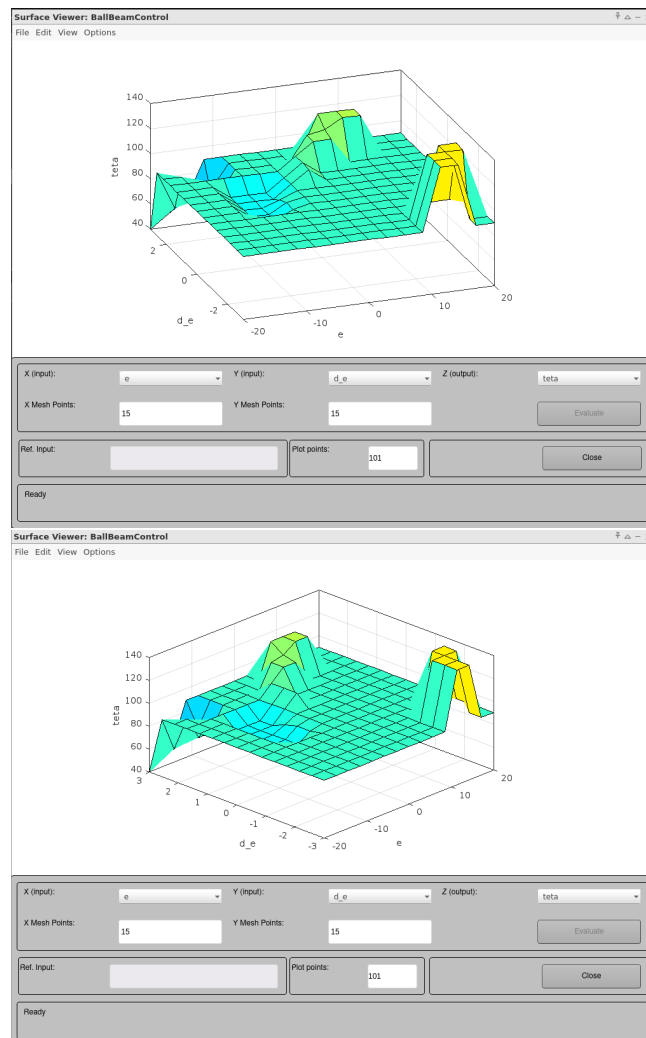
```

U fuzzy logici, pravila su ključna za određivanje kako se ulazi (greška i promjena greške) mapiraju na izlaz (ugao). Svako pravilo u sistemu fuzzy logike daje određeni uslov i odgovarajuću akciju na osnovu pripadnosti funkcijama.

Pravila su definisana na sledeći način:

- `e==NVL & d_e==d_PVL => teta=m_NVL`: Ako je greška `e` veoma negativna (NVL) i promjena greške `d_e` veoma pozitivna (d_PVL), tada ugao `teta` treba biti veoma mali (m_NVL).
- `e==NL & d_e==d_PL => teta=m_NL`: Ako je greška `e` negativna (NL) i promjena greške `d_e` pozitivna (PL), tada ugao `teta` treba biti mali (m_NL).
- `e==NSL & d_e==d_PSL => teta=m_NSL`: Ako je greška `e` prilično negativna (NSL) i promjena greške `d_e` prilično pozitivna (PSL), tada ugao `teta` treba biti prilično mali (m_NSL).
- `e==MN & d_e==d_NS => teta=m_MN`: Ako je greška `e` srednja (MN) i promjena greške `d_e` srednje negativna (NS), tada ugao `teta` treba biti srednje mali (m_MN).
- `e==NS & d_e==d_Z => teta=m_NS`: Ako je greška `e` malo negativna (NS) i promjena greške `d_e` skoro nula (Z), tada ugao `teta` treba biti malo manji od stacionarnog (m_NS).
- `e==Z & d_e==d_PS => teta=m_Z`: Ako je greška `e` skoro nula (Z) i promjena greške `d_e` malo pozitivna (PS), tada ugao `teta` treba biti skoro nula (m_Z).
- `e==PS & d_e==d_MP => teta=m_PS`: Ako je greška `e` malo pozitivna (PS) i promjena greške `d_e` srednje pozitivna (MP), tada ugao `teta` treba biti veći (m_PS).
- `e==MP & d_e==d_PSL => teta=m_PM`: Ako je greška `e` srednje pozitivna (MP) i promjena greške `d_e` prilično pozitivno velika (PSL), tada ugao `teta` treba biti srednje veliki (m_PM).
- `e==PSL & d_e==d_NVL => teta=m_PL`: Ako je greška `e` prilično pozitivno velika (PSL) i promjena greške `d_e` veoma negativna (NVL), tada ugao `teta` treba biti veoma veliki (m_PL).
- `e==PL & d_e==d_NL => teta=m_PL`: Ako je greška `e` velika pozitivna (PL) i promjena greške `d_e` negativna velika (NL), tada ugao `teta` treba biti veoma veliki (m_PL).

Ova pravila su dizajnirana tako da se pokriju različiti scenariji u kojima se greška i promjena greške nalaze. Svako pravilo koristi fuzzy funkcije pripadnosti (membership functions) da bi se odlučilo koji ugao treba primijeniti za različite kombinacije greške i promjene greške.



Slika 29: Vizuelni prikaz veze ulaza i izlaza - Fuzzy upravljanje

Broj Pravila	Ulaz: e	Ulaz: d_e	Izlaz: teta
1	NVL	d_PVL	m_NVL
2	NL	d_PL	m_NL
3	NSL	d_PSL	m_NSL
4	MN	d_NS	m_MN
5	NS	d_Z	m_NS
6	Z	d_PS	m_Z
7	PS	d_MP	m_PS
8	MP	d_PSL	m_PM
9	PSL	d_NVL	m_PL
10	PL	d_NL	m_PL

Table 3: Tabelarni prikaz pravila fuzzy sistema

5.4.1 Kod za ESP32

```

1  #include <ESP32Servo.h>
2  #include <WiFi.h>
3  #include <HTTPClient.h>
4  #include <ArduinoJson.h>
5  #include <vector>
6
7  const char* ssid = "";
8  const char* password = "";
9
10 const int potPin = 34;
11 const int servoPin = 12;
12
13
14 const float beamLengthCM = 40.0;
15 const float sampleTime = 0.1;
16
17 float desiredPosition = 20.0;
18 float previousError = 0.0;
19 float previousPosition = 0.0;
20
21 Servo servo;
22
23 std::vector<float> recordedResponse;
24 std::vector<unsigned long> timeData;
25 int sampleNumber = 0;
26
27 float readBallPosition();
28 void fuzzyController(float position, float velocity);

```

```

29 float computeMembership(float value, float a, float b, float c,
    float d);
30 void sendData(const std::vector<float>& positionData, const
    std::vector<unsigned long>& timeData);
31
32 void setup() {
33     Serial.begin(115200);
34     servo.attach(servoPin);
35     delay(1000);
36
37     WiFi.begin(ssid, password);
38     Serial.println("Connecting to WiFi...");
39     while (WiFi.status() != WL_CONNECTED) {
40         delay(1000);
41         Serial.print(".");
42     }
43     Serial.println("\nConnected to WiFi");
44 }
45
46 void loop() {
47     float currentPosition = readBallPosition();
48     float velocity = (currentPosition - previousPosition) /
        sampleTime;
49
50     fuzzyController(currentPosition, velocity);
51
52     previousPosition = currentPosition;
53
54     recordedResponse.push_back(currentPosition);
55     timeData.push_back(millis());
56     sampleNumber++;
57
58     if (sampleNumber >= 500) {
59         sendData(recordedResponse, timeData);
60         recordedResponse.clear();
61         timeData.clear();
62         sampleNumber = 0;
63     }
64
65     delay(100);
66 }
67
68 float readBallPosition() {
69     int potValue = analogRead(potPin);
70     float positionCM = (potValue / 4095.0) * beamLengthCM;
71     Serial.print("Ball Position (cm): ");
72     Serial.println(positionCM);
73     return positionCM;
74 }
75
76 void fuzzyController(float position, float velocity) {

```



```

77     float error = desiredPosition - position;
78     float deltaError = error - previousError;
79     previousError = error;
80
81     // Izracunavanje stepena pripadnosti za gresku
82     float e_NVL = computeMembership(error, -20, -20, -19.5, -15);
83     float e_NL = computeMembership(error, -19.5, -15, -15, -12.5);
84     float e_NSL = computeMembership(error, -15, -12.5, -12.5, -10);
85     float e_MN = computeMembership(error, -12.5, -9, -9, -5);
86     float e_NS = computeMembership(error, -7.5, -5, -5, -1);
87     float e_Z = computeMembership(error, -1, -0.5, 0.5, 1);
88     float e_PS = computeMembership(error, 1, 5, 5, 7.5);
89     float e_MP = computeMembership(error, 5, 9, 9, 12.5);
90     float e_PSL = computeMembership(error, 10, 12.5, 12.5, 15);
91     float e_PL = computeMembership(error, 12.5, 15, 15, 19.5);
92     float e_PVL = computeMembership(error, 15, 19.5, 20, 20);
93
94     // Izracunavanje stepena pripadnosti za promjenu greske
95     float de_NVL = computeMembership(deltaError, -3, -3, -2.5, -2);
96     float de_NL = computeMembership(deltaError, -2.5, -2, -2, -1.5);
97     float de_NSL = computeMembership(deltaError, -2, -1.5, -1.5,
98         -0.5);
99     float de_MN = computeMembership(deltaError, -1.5, -0.5, -0.5,
100         1.5);
101     float de_NS = computeMembership(deltaError, -0.5, 0.5, 0.5, 1.5);
102     float de_Z = computeMembership(deltaError, -0.5, 0, 0, 0.5);
103     float de_PS = computeMembership(deltaError, 0.5, 1.5, 1.5, 2);
104     float de_MP = computeMembership(deltaError, 1.5, 2, 2, 2.5);
105     float de_PSL = computeMembership(deltaError, 2, 2.5, 2.5, 3);
106     float de_PL = computeMembership(deltaError, 2.5, 3, 3, 3.5);
107     float de_PVL = computeMembership(deltaError, 3, 3, 3.5, 4);
108
109     // Definiranje pravila fuzzy logike
110     float angle_NVL = max(min(e_NVL, de_PVL), min(e_NL, de_PL));
111     float angle_NL = max(min(e_NL, de_PL), min(e_NL, de_PSL));
112     float angle_NSL = max(min(e_NSL, de_MP), min(e_MN, de_PSL));
113     float angle_MN = max(min(e_MN, de_NS), min(e_NS, de_MP));
114     float angle_NS = max(min(e_NS, de_Z), min(e_MN, de_MN));
115     float angle_ZE = max(min(e_Z, de_Z), min(e_PS, de_PS));
116     float angle_PS = max(min(e_PS, de_NS), min(e_MP, de_Z));
117     float angle_PM = max(min(e_MP, de_NSL), min(e_PSL, de_NS));
118     float angle_PL = max(min(e_PSL, de_NVL), min(e_PL, de_NSL));
119     float angle_PVL = max(min(e_PVL, de_NVL), min(e_PL, de_NL));
120
121     // Defuzifikacija koristeći centroidnu metodu
122     float numerator = angle_NVL * 30 + angle_NL * 50 + angle_NSL *
123         60 + angle_MN * 70 + angle_NS * 80 + angle_ZE * 90 + angle_PS
124         * 100 + angle_PM * 110 + angle_PL * 130 + angle_PVL * 150;
125     float denominator = angle_NVL + angle_NL + angle_NSL + angle_MN
126         + angle_NS + angle_ZE + angle_PS + angle_PM + angle_PL +
127         angle_PVL;

```

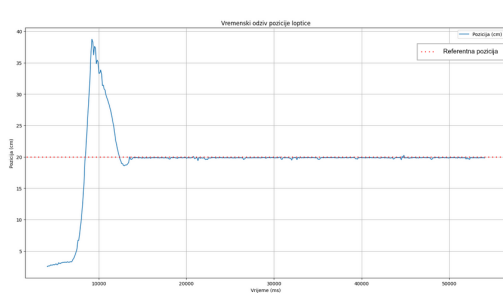
```

122
123     float outputAngle;
124     if (denominator != 0) {
125         outputAngle = numerator / denominator; // Izracunavanje
126             izlaznog ugla
127     } else {
128         // Postavljanje ugla na 90 ako je rezultat nedefinisan
129         outputAngle = 90;
130     }
131
132     if(error >= -20 && error <= -19.5){
133         outputAngle = 30;
134     }
135
136     if(error >= 19.5 && error <= 20){
137         outputAngle = 150;
138     }
139
140     outputAngle = constrain(outputAngle, 0, 180);
141
142     servo.write(outputAngle);
143
144     Serial.print("Control Angle: ");
145     Serial.println(outputAngle);
146 }
147
148 // Funkcija za izracunavanje stepena pripadnosti
149 float computeMembership(float value, float a, float b, float c,
150     float d) {
151     float result = 0.0;
152     if (value >= a && value <= b) {
153         result = (value - a) / (b - a);
154     } else if (value >= b && value <= c) {
155         result = 1.0; // Maksimalna pripadnost
156     } else if (value >= c && value <= d) {
157         result = (d - value) / (d - c); // Opadanje
158     } else {
159         result = 0.0; // Van opsega
160     }
161     // Vracanje stepena pripadnosti
162     return result;

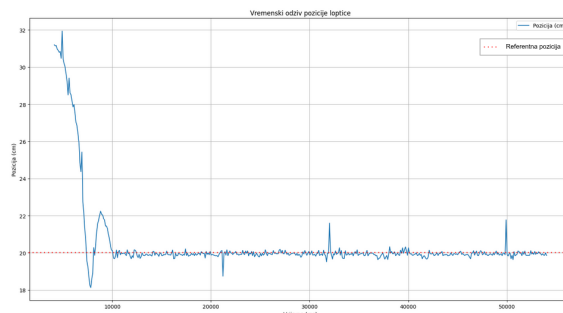
```

5.4.2 Analiza rezultata fuzzy kontrolera na realnom modelu

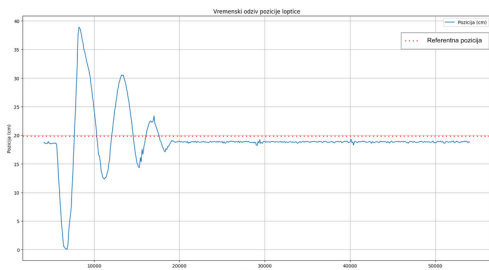
Za prethodno objasnjene ulaze, pravila i izlaze dobijaju se odzivi kao na slikama. Sa slike 29.a)



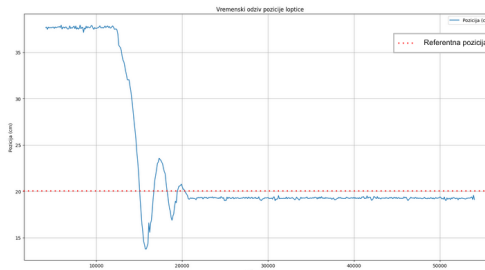
(a) Početni položaj loptice $r(0) = 0$ za granice funkcije pripadnosti Z $(-1, -0.5, 0.5, 1)$



(b) Početni položaj loptice $r(0) = 35$ za granice funkcije pripadnosti Z $(-1, -0.5, 0.5, 1)$



(c) Početni položaj loptice $r(0) = 0$ za granice funkcije pripadnosti Z $(-2.5, -0.5, 0.5, 2.5)$



(d) Početni položaj loptice $r(0) = 35$ za granice funkcije pripadnosti Z $(-2.5, -0.5, 0.5, 2.5)$

Slika 30: Odziv sistema upravljan fuzzy kontrolerom

možemo vidjeti da je vrijeme smirenja približno 5 sekundi, a preskok je 90%, dok je na slici 29. b) maksimalni preskok 10%, a vrijeme smirenja približno 5 sekundi. Tako velike razlike su posljedica načina na koji je namješten kontroler. Ove rezultate je moguće poboljšati drugačijim izborom fukcija pripadnosti.

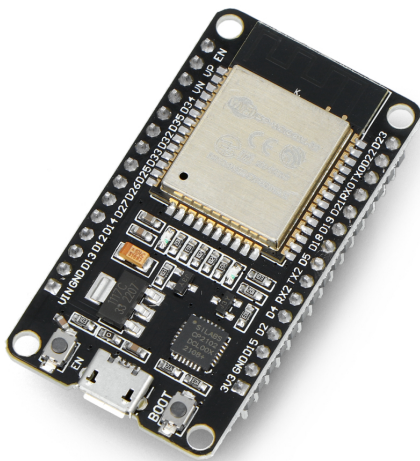
U slučaju kada su za funkciju pripadnosti nultog područja greške (Z) izabrane granice trapezoidne funkcije kao $[-2.5 -0.5 0.5 2.5]$ postojala je veća tolerancija greške. Nastala greška u stacionarnom stanju se sa slike 22. c) i d) može jasno primjetiti i iznosi približno 1[cm].

6 Tehnička implementacija modela

6.1 Korištene komponente

Za realizaciju modela lopte i grede korištene su sljedeće komponente:

- **Servo motor MG996R:** Ovaj servo motor koristi se za upravljanje nagibom grede. Sa maksimalnim zakretnim momentom od 11 kg/cm, omogućava precizno pozicioniranje i stabilnost sistema.
- **SoftPot linearni membranski potencijometar:** Korišten je za merenje pozicije lopte duž grede. Ovaj potencijometar omogućava precizno očitavanje položaja zahvaljujući svojoj tankoj i fleksibilnoj strukturi.
- **Breadboard/Matador ploča:** Ova ploča je korišćena za povezivanje svih elektronskih komponenti bez potrebe za lemljenjem, što je omogućilo lako testiranje i modifikacije u toku razvoja.
- **Power modul:** Obezbeđuje stabilno napajanje za komponente sistema.
- **ESP32 mikrokontroler:** Centralna jedinica za obradu podataka i upravljanje sistemom. ESP32 je korišten za implementaciju kontrolnih algoritama i komunikaciju sa ostalim komponentama.



(a) Mikrokontroler ESP32



(b) Linearni potencijometar



(c) MG996R Servo motor

Slika 31: Korištene komponente

6.2 Očitavanje pozicije pomoću SoftPot linearnog membranskog potencimetra

U okviru ovog modela, za očitavanje pozicije lopte na gredi koristi se SoftPot linearni membranski potencijometar. Potencijometar funkcioniše na principu promjenljivog napona koji zavisi od pozicije lopte duž otporne staze.

SoftPot potencijometar je tanki, fleksibilni senzor koji sadrži sloj otporne staze i sloj kolektora, sa izolacionim slojem između njih. Kada se izvrši pritisak na površinu, ta dva sloja dolaze u kontakt (potrebna je sila od 1N do 3N). Kada se dodirne određena tačka na otpornoj stazi, otpor između te tačke i referentne tačke (najčešće jednog od krajeva potencijometra) se mijenja. Ovaj otpor zavisi od položaja tačke dodira duž staze. Na primjer, ako je tačka dodira blizu jednog kraja, otpor će biti manji u odnosu na drugi kraj. ESP32 mikrokontroler napaja potencijometar sa konstantnim naponom. Kada se vrši pritisak na određenu tačku, napon na srednjem pin-u (wiper) potencijometra se mijenja u zavisnosti od otpora između tačke dodira i referentnog kraja. Ovaj promjenljivi napon je direktno proporcionalan položaju tačke duž staze.

Mikrokontroler ESP32, preko svog analogno-digitalnog konvertora (ADC), očitava napon sa pin-a potencijometra. Ovaj napon je proporcionalan poziciji lopte na gredi, a digitalna vrijednost očitane veličine se dalje koristi za izračunavanje tačne pozicije lopte. Jednačina koja se koristi za pretvaranje očitane digitalne vrijednosti (`potValue`) u poziciju izraženu u centimetrima (`positionCM`) je sljedeća:

$$\text{positionCM} = \left(\frac{\text{potValue}}{4095.0} \right) \times \text{potLengthCM}$$

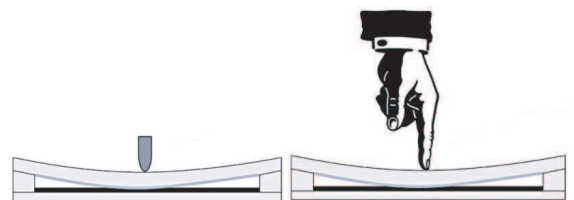
Gdje:

- `potValue` predstavlja digitalnu vrijednost napona očitano sa potencijometra.
- 4095.0 je maksimalna vrijednost koju ADC može očitati u rezoluciji od 12 bita.
- `potLengthCM` je ukupna dužina potencijometra u centimetrima.

Na ovaj način, napon koji potencijometar proizvodi se koristi za precizno određivanje položaja lopte duž grede, što je ključno za uspješnu kontrolu sistema.



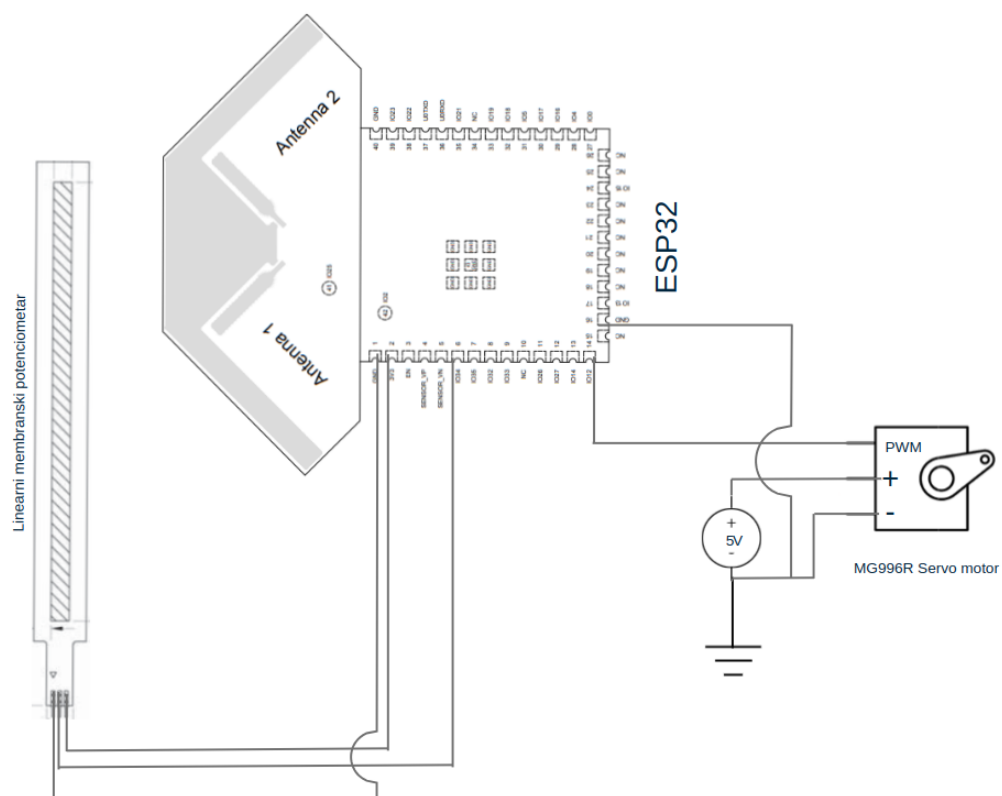
(a) Struktura potencijometra



(b) Princip rada potencijometra

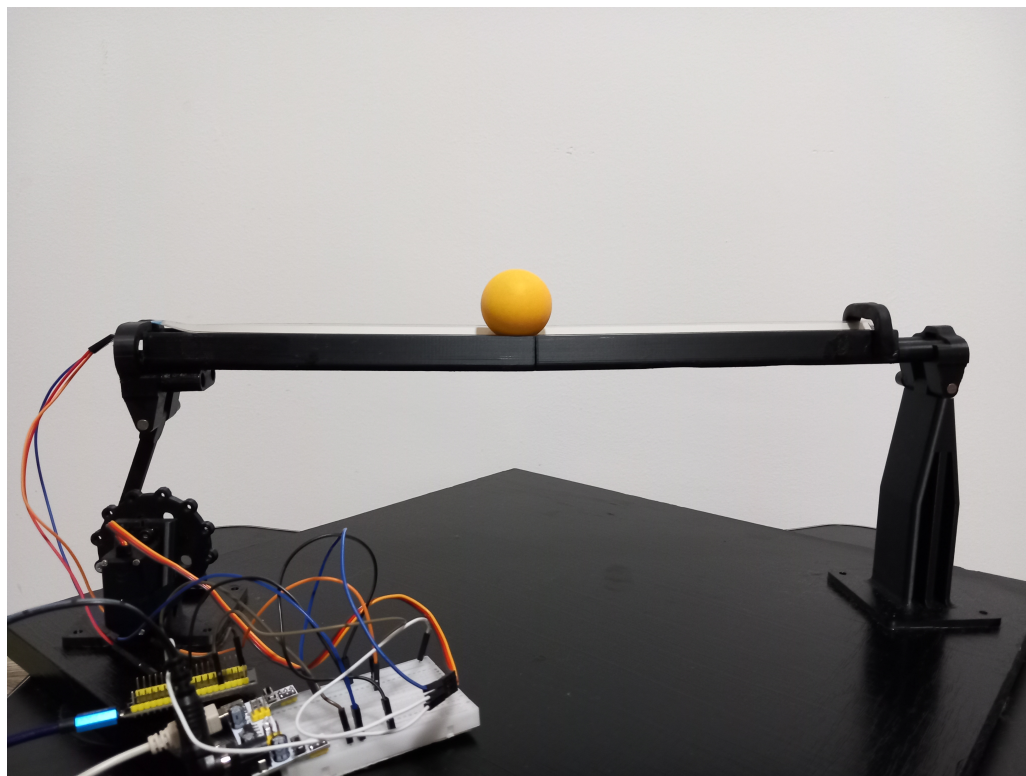
Slika 32: Linearni potencijometar

6.3 Električna šema



Slika 33: Električna šema

6.4 Realizacija sistema



Slika 34: Realizovani model

Pored navedenih komponenti, struktura koja drži i pokreće gredu je 3D printana. Motor je spojen na zupčanik, a na zupčanik je pričvršćena poluga koja pomjera gredu sa ugaonim pomjerajem motora. Lopta je napravljena od plastike i ispunjena olovom i ima masu 190 grama(prilikom simulacija masa je zaokružena na 200 grama).

7 Prikupljanje podataka

Prvi korak u prikupljanju podataka je očitavanje pozicije lopte u realnom vremenu i slanje tih podataka na server pomoću HTTP-a. ESP32 mikrokontroler koristi funkciju `sendData`, koja šalje vektor pozicija i vremenskih oznaka na unaprijed definisanu IP adresu servera putem HTTP POST zahtjeva. Funkcija `sendData` prvo provjerava da li je mikrokontroler povezan na Wi-Fi mrežu. Ukoliko je veza uspostavljena, kreira se HTTP zahtjev koji sadrži pozicije lopte i vremenske oznake. Ti podaci se zatim serijalizuju u JSON format i šalju na server pomoću POST metode.

Na serveru, Python aplikacija koristi Flask framework za prijem podataka. Kada server primi POST zahtjev na ruti `/updatePosition`, on dodaje nove podatke u odgovarajuće liste `position_data` i `time_data`, a zatim generiše grafikon koji prikazuje poziciju lopte u funkciji vremena. Ova funkcija prvo provjerava da li server trenutno prihvata nove podatke (listening flag). Ukoliko je server pauziran, vraća se odgovarajuća poruka. Ako server prima podatke, oni se dodaju u postojeće liste i poziva se funkcija `plot_data`, koja generiše grafikon.

Funkcija `plot_data` koristi biblioteku `matplotlib` za kreiranje grafikona pozicije lopte u funkciji vremena.

7.1 Kod na ESP32

```
1 void sendData(const std::vector<float>& positionData, const
   std::vector<unsigned long>& timeData) {
2     if (WiFi.status() == WL_CONNECTED) {
3         HTTPClient http;
4         http.begin("http://192.168.1.16:5000/updatePosition");
5
6         http.addHeader("Content-Type", "application/json");
7         StaticJsonDocument<400> jsonDoc;
8         JsonArray positions = jsonDoc.createNestedArray("positions");
9         JsonArray times = jsonDoc.createNestedArray("times");
10
11        for (float position : positionData) {
12            positions.add(position);
13        }
14
15        for (unsigned long timeStamp : timeData) {
16            times.add(timeStamp);
17        }
18
19        String jsonString;
20        serializeJson(jsonDoc, jsonString);
21
22        int httpResponseCode = http.POST(jsonString);
23        http.end();
24    }
25 }
```

7.2 Server kod


```

1 from flask import Flask, request, jsonify
2 import matplotlib.pyplot as plt
3 import threading
4 import sys
5 import time
6
7 app = Flask(__name__)
8
9 position_data = []
10 time_data = []
11
12 listening = True
13
14 @app.route('/updatePosition', methods=['POST'])
15 def update_position():
16     global position_data, time_data, listening
17
18     if not listening:
19         return jsonify({"message": "Server je pauziran."}), 403
20
21     # Primanje JSON podataka POST request-om
22     data = request.get_json()
23     if 'positions' in data and 'times' in data:
24         position_data.extend(data['positions'])
25         time_data.extend(data['times'])
26         plot_data()
27         # Odgovor porukom da su uspjesno primljeni podaci
28         return jsonify({"message": "Podaci uspjesno primljeni"}), 200
29     else:
30         return jsonify({"message": "Nepravilan format poruke"}), 400
31
32
33 def plot_data():
34     global position_data, time_data
35
36     if len(position_data) == 0 or len(time_data) == 0:
37         return jsonify({"message": "Nema podataka za iscrtavanje"}),
38             400
39
40     # Plot data
41     plt.figure()
42     plt.plot(time_data, position_data, label='Pozicija (cm)')
43     plt.xlabel('Vrijeme (ms)')
44     plt.ylabel('Pozicija (cm)')
45     plt.title('Vremenski odziv pozicije loptice')
46     plt.grid(True)
47     plt.legend()
48     plt.show()
49
50     return jsonify({"message": "Generisan grafik"}), 200

```

```

50
51 def pause_and_resume_server():
52     global listening
53
54     input("Pritisni enter za iscravanje grafika...")
55     listening = False
56
57     # Generisanje grafika
58     with app.test_request_context('/plotData', method='GET'):
59         app.dispatch_request()
60
61     input("Enter za nastavak rada servera...")
62     listening = True
63     print("Server nastavlja sa radom")
64
65 if __name__ == '__main__':
66     threading.Thread(target=pause_and_resume_server,
67                       daemon=True).start()
68     app.run(host='0.0.0.0', port=5000)

```

8 Zaključak

U ovom radu uspješno je realizovan ball and beam sistem koristeći različite linearne i nelinearne metode upravljanja. Eksperimentalna analiza pokazala je da svaki od implementiranih kontrolera (PID, fuzzy logika i kontroler sa povratnom spregom po cijelom vektoru stanja) ima svoje prednosti i mane u zavisnosti od specifičnih zahtjeva sistema.

PID kontroler je pokazao brz odziv na step ulaz, ali uz veći maksimalni preskok, što može biti nepovoljno u aplikacijama gdje je stabilnost ključna. Fuzzy logika se pokazala kao najfleksibilniji pristup, omogućavajući bolje upravljanje nelinearnostima sistema, iako zahtijeva pažljivu definiciju pravila i funkcija pripadnosti. Kontroler sa povratnom spregom po cijelom vektoru stanja obezbijedio je stabilnost sistema, ali uz određene izazove u implementaciji zbog potrebe za preciznim modelovanjem sistema.

Korištenje MATLAB-a za simulaciju i optimizaciju kontrolera značajno je doprinijelo uspješnu implementaciju na stvarnom sistemu, omogućavajući preciznu analizu performansi prije fizičke realizacije.

Na osnovu ovih rezultata, može se zaključiti da je izbor odgovarajućeg kontrolera ključan i da zavisi od specifičnih zahtjeva primjene.

Reference

- [1] Carlos G. Bolívar-Vincenty Gerson Beauchamp-Báez. Modelling the ball-and-beam system from newtonian mechanics and from lagrange methods.
- [2] Marta Virseda. Modeling and control of the ball and beam process.
- [3] Control Tutorials for Matlab and Simulink. <https://ctms.engin.umich.edu/ctms/?example=ballbeam>
- [4] PID controller github vastevenson. <https://github.com/vastevenson/tilt-pid-controller-arduino>.
- [5] 3D model. <https://grabcad.com/library/ball-and-beam-2>.